

Quantum-Inspired Manta Ray Foraging Optimization for Cloud Task Scheduling

An implementation-first research investigation (report, 22 September 2026)

Scope. Quantum-*inspired* classical optimization for cloud computing. Every experiment in this report ran on an ordinary laptop CPU (Windows 11, Python 3.13 / NumPy 2.3 for the pilots; a Docker image with Python 3.12 for the notebook). No quantum hardware, quantum circuits, QPU or annealer was used or is required.

Rules. Every number labelled *measured* was produced by the code in this repository (scripts/observe/observe_v0.py ... observe_v3.py, scripts/observe/observe_dyn*.py, the notebooks, and the V5 experiments in experiments/). Anything not measured is labelled UNMEASURED; illustrative numbers are labelled HYPOTHETICAL. All literature statements come from pages opened by live web search on 22 September 2026 (see docs/literature/), never from memory.

Deliverables. notebooks/research/quantum_inspired_cloud_scheduler.ipynb (18 sections, runs top-to-bottom), the Colab notebooks in notebooks/colab/, this report, README.md, docs/lab_log.md, docs/research_plan_v5.md, docs/proposal/phd_research_proposal.md, docs/literature/*.md, results/.

Status after V5 (23 September 2026). Read this first. Sections 1–26 record the V1–V4 investigation as it happened. Sections 27–39 add the V5 held-out tests, which sharpen some earlier claims and overturn others. The current conclusions are in *Final decision* → *V5 update*. In short:

- **Confirmed.** The quantum-inspired register representation repairs DMO and MRFO: on held-out problems and in every run up to 5 000 tasks.
- **Revised.** The Born rule is not needed: the classical linear twin is as good, and slightly better with the swap move (§27).
- **Superseded.** The V4 statement that Max-Min beats the swarm on static batches (§26). With the swap move (CXM), QI-MRFO beats Max-Min on 6 of 8 held-out families, and started from the Max-Min schedule it beats Max-Min in every run at scale (§27, §38).
- **Limit.** For one-time scheduling a simple (1+1)-EA with the same moves does as well (§29). The swarm's own advantage is re-scheduling a changing cloud under a migration price, where the final configuration beats the best heuristic strategy at every price tested (§37).
- **Falsified or rejected.** Purity-regulated decoherence (§32), the unconditional elite (§31), and the swap move in QI-DMO (§39).

1. Selected algorithm

MRFO (Manta Ray Foraging Optimization; Zhao, Zhang & Wang, 2020) is the primary host of the quantum-inspired mechanism (QI-MRFO). **DMO (Dwarf Mongoose Optimization; Agushaka, Ezugwu & Abualigah, 2022)** is implemented as a second host (QI-DMO) because the mechanism is host-agnostic and DMO is the algorithm of the candidate's prior project; showing the transfer is part of the evidence. PSO was not selected.

2. Why it was selected (Phase 3)

The choice was made *after* the literature mapping and *after* the first pilot, on the criteria requested:

Criterion	PSO	DMO	MRFO
Weakness of the original optimizer on cloud scheduling (measured, §17–18)	degrades with n (gap 21 % at n=100)	random-search level for n $\geq$ 50 (gap 76 %)	random-search level for n $\geq$ 50 (gap 98 %); collapses diversity
Literature gap for a quantum-inspired version	saturated: $\geq$ 20 QPSO/QI-PSO papers incl. cloud (docs/literature/qpso_literature_map.md)	5 quantum-labelled papers, all binary Q-bit feature selection or a relabelled Gaussian jump; none for scheduling	6 quantum-labelled items, all continuous, none discrete, none for scheduling; the two MRFO surveys each index a single one
Suitability for the amplitude representation	fine, but already done (Balicki 2022, Jeong 2010)	natural mapping (babysitter reset = full decoherence)	natural mapping (best-centred moves become perturbations of the best basis state; somersault = reflection about it)
Cloud-scheduling relevance	high	growing (Abraham/Ngadi line 2025–26)	~10 papers, mostly regional; one 2025 study shows hybrid MRFO losing to PSO-GA
Discrete compatibility	needs transfer functions	needs transfer functions	needs transfer functions (Yıldızdan 2023; Bouaita 2026)
Implementation feasibility	done	done (MATLAB/MEALPY-faithful)	done (MEALPY-faithful)
Measured improvement from the mechanism (pilot)	UNMEASURED (not implemented)	76 % $\rightarrow$ 3.9 % gap at n=100; 1.4 % with a greedy next-position step	98 % $\rightarrow$ 1.0 % gap at n=100; ties/beats the discrete GA on all four pilot instances
Identity preservation	—	preserved, but the original's unconditional "next position" move is a handicap	preserved; greedy replacement in every phase makes the mechanism most effective

MRFO is primary because the measured effect is largest and cleanest and the literature gap is documented by a 2025 survey; DMO is secondary because the candidate's background is DMO and because the transfer result strengthens the science (the mechanism is representational, not host-specific). PSO was excluded because quantum-inspired PSO is the most crowded corner of the literature and a Q-bit PSO for VM placement already exists (Balicki, Entropy 2022).

3. Existing quantum-inspired versions (Phase 1)

Full tables with URLs: docs/literature/qpso_literature_map.md (23 papers), docs/literature/dmo_literature_map.md (5 quantum DMO papers + 9 cloud DMO papers), docs/literature/mrfo_literature_map.md (6 quantum MRFO items + 12 cloud MRFO papers), docs/literature/qi_cloud_and_classical_equivalents.md (19 quantum-inspired cloud schedulers + the mechanism-without-the-word-quantum analysis). Highlights:

- **QPSO** (Sun, Feng & Xu 2004; Sun et al. 2011/2012): $x = p \pm \alpha |mbest - x| \ln(1/u)$. Its authors state it is a bare-bones PSO sampling a double-exponential distribution; Mikki & Kishk (2006) show the delta well is a Laplace kernel. Cloud uses (Yu et al. 2021 HWQPSO; Wang et al. 2023 DE3C; Elsedimy 2025; Aminu 2026) round the continuous position to a VM index and evaluate static batches; the Naik/Bey group replaces the sampler by Q-bits + hashing (an EDA). No verified QPSO cloud paper tests dynamic workloads.
- **Quantum DMO**: DMOAQ (Abd Elaziz et al., Mathematics 2022: binary Q-bit, Han–Kim rotation table, threshold measurement, DMO operators on the angle vector); three papers copy its equations into deep-learning pipelines (Almutairi 2023; Deepa 2024; Sivakumaran 2025); EDMO (arXiv 2511.09020, Nov 2025) relabels a fitness-scaled Gaussian jump as "quantum tunnelling". None is multi-valued, none is for scheduling.
- **Quantum MRFO**: Gupta & Kumar (IJCSSE 2024, Bloch-phase encoding, 20 continuous benchmarks); Turgut et al. (Sci Rep 2025, QPSO delta-well move on the worst third of the population); Wang et al. (Biomimetics 2023, Bloch-coded initialization only); QFLN-QIMRO (J. Supercomputing, 20 Sep 2026, quantum-behaved MRFO for 6G MEC orchestration; equations paywalled); two unreadable items (Ramadan 2021; CRC 2022 chapter). None discrete, none for cloud task/workflow/VM scheduling, none with code.
- **Quantum-inspired cloud schedulers on other hosts** (2021–2026): QIGA/QIEA with Q-bit chromosomes and rotation gates (Hussain 2022/2024; Lilhore 2025 QHRMOF; Hammouda 2026 FOG-QIEA; Chauhan & Alam 2024; Galavani 2025), Q-bit PSO for VM placement (Balicki 2022), Q-bit DE (Priyanka 2024), RQWOA (Divya 2026), and two "quantum" heuristics with no probabilistic structure at all (QBDS 2026, QIARM 2025). A 2026 arXiv benchmark of QUBO/quantum-inspired HPC workflow mapping (Sharma et al.) finds classical solvers preferable.

4. Literature comparison (the quantum-innovation map, condensed)

Family	Quantum mechanism	Classical equivalent	Representation for scheduling	Genuinely different mechanism?	What it does NOT solve
QPSO (delta well)	wave-function sampling around an attractor	Laplace mutation with population-spread scale (authors' own statement)	continuous + rounding	No (kernel choice)	attractor collapse; lossy discretisation; dynamics untested
Q-bit QIEA/QIGA/QI-PSO/QI-DE	qubit amplitudes + rotation gate + measurement	PBIL/cGA/UMDA with a self-damping learning rate (Platel, Schliebs & Kasabov 2007/2009)	binary Q-bits, hashing/random keys	No (univariate EDA)	needs a decoder; no theory; no dynamics
"Quantum tunnelling" variants	jump probability from a pseudo-potential	Cauchy/Lévy mutation (Yao 1999; Lee & Yao 2004)	continuous	No	discrete problems
Quantum-behaved MRFO/DMO (Turgut 2025; QFLN-QIMRO 2026; EDMO 2025)	QPSO move or tunnelling jump inside the host	Laplace/Gaussian jumps	continuous	No	scheduling; discrete encodings

Family	Quantum mechanism	Classical equivalent	Representation for scheduling	Genuinely different mechanism?	What it does NOT solve
Bloch-phase MRFO (Gupta & Kumar 2024)	phase-angle encoding, three real points per individual	reparameterised continuous search	continuous	Partial (representation)	discrete; applications
Simulated quantum annealing / quantum walks (physics literature)	transverse-field replica coupling; ballistic spreading	parallel tempering; random walk	binary/QUBO	Partial (different classical dynamics)	not used by any cloud paper; QUBO mapping loses feasibility beyond ~15 tasks (Sharma 2026)
This work	m-ary amplitude registers per task, Born measurement, basis-state attractors, depolarising channel, swarm difference-vector updates	probability-vector swarm with a mutation floor (classical twin implemented and tested)	native task → VM registers, no decoder	Partial: the <i>representation</i> is what matters; squaring and sign are measurably irrelevant (§19)	static heavy-tailed batches (Max-Min wins); very severe change

5. Existing classical equivalents (Phase 1B, mandatory)

For every quantum mechanism the same idea was searched without the word "quantum" (docs/literature/qi_cloud_and_classical_equivalents.md, Part 2):

Quantum mechanism	Classical equivalent	Who showed it	Mathematical difference
Q-bit rotation gate	PBIL probability-vector update, $\mathbf{p}' = \sin^2(\theta + \Delta\theta) \approx \mathbf{p} + 2\sqrt{\mathbf{p}(1-\mathbf{p})} \Delta\theta$	Platel, Schliebs & Kasabov 2007, 2009 ("QEA is an EDA"); da Silva & Schirru 2014 ("quantum PBIL")	none for sampling; a self-damping learning-rate schedule
Tunnelling	Cauchy / Lévy mutation, restarts	Yao, Liu & Lin 1999; Lee & Yao 2004	none in metaheuristics; partial for path-integral SQA (Santoro 2002; Isakov 2016)
Quantum walk	random walk; classical wave simulation	Knight, Roldán & Sipe 2003; Xue & Sanders 2013	ballistic vs diffusive spreading is a classical interference effect; decoherence restores the random walk
Decoherence / depolarising channel	Baluja's mutation shift $\mathbf{p} \leftarrow (1 - \lambda)\mathbf{p} + \lambda \mathbf{rand}$; Yang & Yao's central-vector mixing; Cobb's hypermutation; Branke's memory	Baluja 1994; Cobb 1990; Branke 1999; Yang & Yao 2005/2008	none on product states (diagonal of $\mathbf{p}$ is all that sampling sees)
Superposition + measurement	sampling a product distribution: UMDA/PBIL/cGA; EDAs already used for cloud (Chen 2014) and fog (Wu, Li, Wang & Zomaya 2018)	—	none
Born rule	squared reparameterisation of a probability vector	Glasser et al. 2019 (separations only for entangled Born machines)	none for product states
Purity floor	cGA margins $[1/n, 1 - 1/n]$; Han & Kim's Hε gate	Doerr & Zheng 2020; Han & Kim 2004	none

Conclusion of Phase 1B: in the cloud-scheduling corpus every quantum-inspired operator is a relabelled EDA/PBIL, Lévy mutation or mutation shift. The genuinely distinct physics-derived algorithms (SQA, quantum walks, entangled Born machines) are not used by any cloud paper — and our own experiments (§19) confirm the same verdict for our mechanism: the useful content is representational.

6. The weakness we target (OBSERVATION)

Measured in the V0 pilot (observe_v0.py, 5 seeds, 20 000 evaluations, floor encoding $\mathbf{a}_i = \lfloor \mathbf{x}_i \rfloor$):

Instance	LB	Max-Min	PSO	DMO	MRFO	GA (discrete)	Random
n=30, m=5, uniform tasks, heterogeneous VMs	44.53	45.46	45.71 (2.7 %)	45.73 (2.7 %)	47.64 (7.0 %)	44.87 (0.8 %)	47.03 (5.6 %)
n=50, m=10, bimodal (heavy-tailed) tasks	25.11	27.06	28.75 (14.5 %)	36.50 (45.4 %)	35.45 (41.2 %)	28.74 (14.5 %)	34.41 (37.1 %)
n=100, m=10, uniform	47.06	47.45	56.86 (20.8 %)	82.62 (75.6 %)	93.39 (98.5 %)	47.72 (1.4 %)	90.22 (91.7 %)
n=50, m=10, homogeneous VMs	28.39	28.98	30.47 (7.3 %)	33.04 (16.4 %)	33.26 (17.2 %)	29.37 (3.5 %)	32.90 (15.9 %)

(makespan in seconds; brackets = gap to the lower bound.) The instrumentation shows why: at n=100 a DMO candidate changes on average **48.9** task assignments, an MRFO candidate **43.9**, a PSO candidate 13.8, a GA child 5.4; only 1.4 % (MRFO) and 2.8 % (PSO) of candidates improve; MRFO's population diversity (mean pairwise Hamming distance / n) collapses to 0.02 while 15 % of its evaluations are spent on candidates identical to the parent. The landscape probe shows that improving moves on this landscape change 1–5 tasks (90–100 % of 1-move local optima are improvable by a 2-task swap). **The floor encoding gives the swarm no controllable notion of move size**: steps are proportional to inter-individual distances in $[0, m]^n$ and become random re-draws after rounding. This is not a parameter problem (PSO shares the trend) and not a metaphor problem (the same algorithms are competent on continuous functions); it is the representation.

7. Our quantum-inspired mechanism (MECHANISM)

- Representation.** Individual i holds $\Psi_i \in \mathbb{R}^{n \times m}$: one m -level real amplitude register (a "rebit" generalised to m outcomes) per task, rows on the unit sphere $\sum_j \Psi_i[t, j]^2 = 1$. Initial state: the uniform superposition $m^{-1/2} \mathbf{1}$.
- Measurement (Born rule).** $\Pr(\mathbf{a}_t = j) = \Psi[t, j]^2$, tasks sampled independently; one measurement = one evaluation; a measured schedule $\mathbf{b}$ is encoded back as the basis state $\mathbf{E}(\mathbf{b})$ (one-hot rows).
- Host dynamics unchanged.** Every MRFO (or DMO) equation is applied to the amplitude matrices, followed by the projection Π (row renormalisation). Attractors are basis states: $\mathbf{E}(\mathbf{b}_{best})$ for MRFO, $\mathbf{E}(\mathbf{b}_\alpha)$ for DMO ("the alpha peeps its measured food position").
- Decoherence.** Each candidate register passes through the depolarising channel $\mathcal{D}_\gamma(p) = (1 - \gamma)p + \gamma/m$ (amplitudes $\leftarrow \text{sign}(\Psi) \sqrt{\mathcal{D}_\gamma(\Psi^2)}$), $\gamma = c/n$. It bounds purity away from 1 and therefore guarantees a floor on the expected number of reassigned tasks per measurement. With $\gamma = 1$ it is the classical random reset (DMO's babysitter exchange); intermediate γ is *controlled forgetting*.
- Purity as the move-size dial.** $\pi_t = \sum_j p_{tj}^2$; the expected number of tasks that differ between two measurements of the same register set is $n - \sum_t \pi_t$. The host's own dynamics move purity

continuously: attraction to a basis state raises it (exploitation), mixing with a diffuse register or a random basis state lowers it (exploration).

6. **Structural rules for dynamic environments.** VM removal = delete the column and renormalise (a projective measurement onto the surviving VMs); VM addition = a new column with the uniform share $1/(m+1)$; a replaced task = its register reset to the uniform superposition.

8. Original equations

MRFO ($r, r_1, r_2, r_3 \sim U(0, 1)$; t = iteration, T = iteration budget; greedy replacement after each phase; $i = 1$ uses x_{best} as predecessor):

- Chain: $x_i^{t+1} = x_i^t + r(x_{i-1}^t - x_i^t) + \alpha(x_{best} - x_i^t)$, $\alpha = 2r\sqrt{|\log r|}$.
- Cyclone: $x_i^{t+1} = x_{best} + r(x_{i-1}^t - x_i^t) + \beta(x_{best} - x_i^t)$, $\beta = 2e^{r_1(T-t+1)/T} \sin(2\pi r_1)$; while $t/T < \text{rand}$, a uniformly random x_{rand} replaces x_{best} .
- Somersault: $x_i^{t+1} = x_i^t + S(r_2 x_{best} - r_3 x_i^t)$, $S = 2$.

DMO (MATLAB/MEALPY form, which is what the field actually runs; the paper-vs-code discrepancies are documented in docs/literature/dmo_literature_map.md): alpha selection by roulette on $e^{-f_i/\bar{f}}$; alpha-group candidate $X_\alpha + \varphi \odot (X_\alpha - X_k)$, $\varphi \sim \frac{peep}{2} U(-1, 1)^n$; scout candidate $X_i + \varphi \odot (X_i - X_k)$ with sleeping mound $sm_i = (f_{cand} - f_i) / \max(f_{cand}, f_i)$; babysitter exchange (uniform re-initialisation of the first B mongooses when $C_i \geq L = 0.6nB$); next position $X_i \mp CF \varphi r (X_i - sm_i)$, $CF = (1 - t/T)^{2t/T}$, applied unconditionally.

Symbols: x_i position of individual i (one coordinate per task); x_{best} best position; r -type factors are uniform random scalars; $\alpha, \beta, S, \varphi, CF$ are the step-size coefficients defined by the respective papers.

9. Modified equations (what changed and why)

Replace every position by a register matrix and every attractor by a basis state; wrap each update in $\mathcal{D}_\gamma \Pi[\cdot]$; measure the candidate to obtain the schedule that is evaluated:

- Chain: $\Psi_{cand} = \mathcal{D}_\gamma \Pi[\Psi_i + r(\Psi_{i-1} - \Psi_i) + \alpha(E(b_{best}) - \Psi_i)]$.
- Cyclone: $\Psi_{cand} = \mathcal{D}_\gamma \Pi[E(b_{best}) + r(\Psi_{i-1} - \Psi_i) + \beta(E(b_{best}) - \Psi_i)]$ (exploration: $E(b_{rand})$, the basis state of a uniformly random schedule).
- Somersault: $\Psi_{cand} = \mathcal{D}_\gamma \Pi[\Psi_i + S(r_2 E(b_{best}) - r_3 \Psi_i)]$.
- Acceptance: measure $a_{cand} \sim \Psi_{cand}$; if $f(a_{cand}) < f_i$ adopt $(\Psi_{cand}, a_{cand}, f(a_{cand}))$.
- DMO: alpha group $\mathcal{D}_\gamma \Pi[E(b_\alpha) + \varphi \odot (E(b_\alpha) - \Psi_k)]$; scouts $\mathcal{D}_\gamma \Pi[\Psi_i + \varphi \odot (\Psi_i - \Psi_k)]$; babysitter exchange $\Psi_i \leftarrow \mathcal{D}_{\gamma_{reset}}(\Psi_i)$; next position $\mathcal{D}_\gamma \Pi[\Psi_i \mp CF \varphi r (\Psi_i - sm_i)]$; φ drawn per task (per row).

What changed: (i) the state space ($[0, m]^n \rightarrow$ product of n unit spheres in $\mathbb{R}^m$); (ii) attractors are collapsed schedules, not raw states; (iii) a projection and a noise channel after every update; (iv) schedules are sampled, not decoded. What did *not* change: the operators, the control flow, the number of evaluations per iteration, the greedy acceptance — the algorithm is still recognisably MRFO/DMO. Complexity: $O(nm)$ per candidate (update + measurement) instead of $O(n)$; memory Pnm floats.

State-transition process: $(\Psi_i, b_i, f_i) \xrightarrow{\text{operator}} \tilde{\Psi} \xrightarrow{\Pi, \mathcal{D}_\gamma} \Psi_{cand} \xrightarrow{\text{measure}} a_{cand} \xrightarrow{f} f_{cand} \xrightarrow{\text{greedy}} (\Psi'_i, b'_i, f'_i)$.

10. Why the mechanism is genuinely quantum-inspired

The three ingredients are the textbook objects of quantum measurement theory: a normalised amplitude vector per subsystem (a product state over n subsystems), the Born rule that turns amplitudes into outcome probabilities by squaring, and the depolarising channel $\rho \rightarrow (1 - \gamma)\rho + \gamma I/m$, the standard model of decoherence. Purity $\text{Tr } \rho^2$ is the standard measure of how mixed a state is, and the observation that *purity controls the expected Hamming distance between two measurements* is the quantum-information reading of the move-size problem. Signed amplitudes admit cancellation when registers are mixed (interference). The mapping is complete and teachable, and it is *not* decoration: it produced a testable prediction (move size follows purity; a decoherence floor is needed; the optimal γ is small) that the experiments confirmed. What we also report honestly is that the quantum-specific parts (squaring, sign) turned out to be measurably irrelevant (§19), which is exactly the kind of result Phase 1B predicts for product states.

11. Why this is NOT quantum computing

No joint state of n qubits is ever formed (that would need m^n amplitudes); the registers are independent, real, m -dimensional unit vectors updated by ordinary arithmetic; no unitary evolution, no entanglement, no interference across subsystems is simulated; "measurement" is numpy sampling. The whole optimizer is $O(Pnm)$ memory and runs in seconds on a laptop. Nothing here would run faster on a quantum computer, and nothing here requires one.

12. Scientific hypothesis (stated before the experiments, Phase 4/5)

- **Observation.** Floor-encoded DMO/MRFO change $\sim n/2$ tasks per candidate and perform at random-search level for $n \geq 50$ (§6).
- **Mechanism.** Amplitude registers + Born measurement + basis-state attractors + decoherence floor (§7).
- **Causal reasoning.** With registers, the number of tasks changed per candidate is $n - \sum_t \pi_t$, a continuous function of the state that the host's attraction/repulsion dynamics move; therefore the host's exploration–exploitation schedule (which is meaningless after rounding) becomes an effective move-size schedule on the assignment lattice.
- **H1 (encoding).** QI-X reaches lower makespans than X at equal evaluation budgets, with the gain increasing with n and m ; QI-X approaches the discrete GA.
- **H2 (decoherence).** Without the channel the registers collapse (purity $\rightarrow 1$) and evaluations are wasted re-measuring the same schedule; a weak channel ($c \approx 0.25$ –1) removes the waste and improves results; strong decoherence ($c \geq 2$) degrades monotonically (dose–response).
- **H3 (classical twin).** If the benefit is representational, the linear-probability twin captures it; if the Born rule matters, the twin is worse.
- **H4 (dynamic).** After a workload change, carrying the register state beats a restart; a decoherence shock $\gamma_{shock} \in (0, 1)$ ("controlled forgetting") beats both naive continuation and restart, and the optimal γ_{shock} grows with change severity.
- **Falsification.** H1 fails if QI-X does not beat X on $n \geq 50$ or is dominated by the GA by a wide margin; H2 fails if $c = 0$ is best or if performance is flat in c ; H3 is decided either way; H4 fails if no $\gamma_{shock} \in (0, 1)$ beats plain continuation.
- **Anticipated effect size (HYPOTHETICAL, written before the run):** 20–40 % lower makespan than classical DMO/MRFO at $n = 100$ and a gap to the GA below 5 %; 5–15 % better post-change quality for the shock strategy on moderate churn.

13. Expected behaviour BEFORE the experiments

Move size decays smoothly from $\sim n/2$ to a few tasks as purity rises; improving-candidate fraction rises from $\sim 1\%$ to $\geq 10\%$; diversity collapses *unless* the channel is on; QI-MRFO and QI-DMO converge to within a few percent of the lower bound; the linear twin close behind; the register attractor variant (using the alpha's superposition instead of its measured schedule) expected to be worse because nothing concentrates the registers.

14. What result would support the hypothesis

Consistent improvement over the classical host on all instances with the gain growing with n ; a U-shaped or monotone dose–response in c with an interior optimum; wasted-evaluation fraction dropping to ~ 0 with the channel; and, for H4, a strategy with $0 < \gamma_{shock} < 1$ dominating both restart and continuation with the optimum shifting with severity.

15. What result would falsify it

No gain at $n \geq 50$; a flat response in c ; the twin performing differently in the direction that contradicts H3; no interior optimum in γ_{shock} ; or the discrete GA / Max-Min dominating everywhere (which would make the contribution "fixing a broken baseline" rather than a competitive scheduler).

16. First implementation (V1 $\rightarrow$ V3, what exists and how it was built)

All code is pure NumPy/SciPy and lives in three modules that the notebook embeds verbatim (build_notebook.py slices them by section markers, so notebook and modules cannot drift):

- `qi_core.py` — instance model (`CloudInstance`, `make_instance`), objective with evaluation counting (`Objective`), the floor decoder, instrumentation (`Tracker`: best-so-far, diversity, per-candidate move size, wasted/neutral/improving/worse classification, global-best improvements), Min-Min/Max-Min heuristics, a 1-move hill climber for landscape probes, and the classical optimizers PSO, DMO (faithful to the authors' MATLAB code as ported by MEALPY, including the unconditional next-position move; `greedy_next=True` gives the MEALPY-developed variant), MRFO (faithful to MEALPY's `OriginalMRFO`), a discrete GA and random search. Every optimizer accepts and returns a `state` for warm-started re-optimisation.
- `qi_quantum.py` — the mechanism (`born_probs`, `measure`, `basis_state`, `project`, `purity`, `depolarise`, structural-change operators `remove_vm_state`, `add_vm_state`, `add_tasks_state`, `shock_state`) and the two hosts `run_qimrfo`, `run_qidmo` with `mode` $\in$ {`born_signed`, `born_abs`, `linear`}, decoherence (γ), attractor $\in$ {`basis`, `register`}, `gamma_reset`, `greedy_next`.
- `qi_dynamic.py` — dynamic environments (task churn, VM failure, VM addition, speed drift) and the strategies `restart` / `continue` / `continue_struct` / `shock` / `hypermutation`.

Version history (each version has one evidence-driven change; see `lab_log.md`):

Version	Change	Reason (evidence)
V0	classical PSO/DMO/MRFO/GA with floor encoding + instrumentation	OBSERVE
V1	amplitude registers, Born measurement, basis-state attractor, $\gamma_{reset} = 1$	move size uncontrollable (V0)

Version	Change	Reason (evidence)
V2	per-candidate depolarising channel $\gamma = c/n$; transfer to MRFO	26–76 % wasted evaluations after collapse (V1)
V3	ablations: greedy next-position for DMO, register attractor, c sweep for MRFO, linear twin, unsigned, population size	QI-DMO < QI-MRFO (V2): is it the host's unconditional move?; is the Born rule doing anything?
Dyn	warm-start harness, structural rules, shock, severity sweep	H4

The notebook's Section 10 runs 18 validation tests (objective vs brute force, decoder bounds, chi-square test that measurement follows the Born rule, channel normalisation and monotone purity, strict evaluation-budget accounting for every optimizer, determinism, $\gamma = 1$ reduces to uniform sampling, valid schedules in all modes). One bug was found by these tests and fixed: DMO/QI-DMO exceeded the budget by the babysitter re-initialisation evaluations (about 0.1 % in the pilots; the pilot numbers below are therefore, for DMO/QI-DMO, a few evaluations above budget — immaterial, but recorded).

17. Pilot methodology (Phase 9)

- Instances (generator seed 1): (n=30, m=5, uniform tasks U(1000, 10000) MI, heterogeneous VMs 250–2000 MIPS), (50, 10, bimodal heavy-tailed tasks, heterogeneous), (100, 10, uniform, heterogeneous), (50, 10, uniform, homogeneous VMs). Same instance for every algorithm and seed.
- Budget: 20 000 objective evaluations per run for every algorithm (each measurement = one evaluation); population 30; 5 run seeds (0–4); objective = makespan; results reported as mean makespan and gap to the Q||Cmax lower bound; Max-Min and Min-Min as heuristic references; random search as the floor.
- Controlled initial conditions: all population methods start from the uniform distribution over schedules (uniform positions / uniform superposition / uniform integers).
- Mechanism metrics: per-candidate move size (tasks changed vs parent), wasted/neutral/improving/worse fractions, global-best improvements per 1 000 evaluations, population diversity (mean pairwise Hamming/n), register purity, runtime.
- Notebook `full` mode repeats everything with 30 seeds and adds held-out instance shapes (n=200 m=20; lognormal tasks with low heterogeneity; bimodal on homogeneous VMs).

18. Experimental results (measured; pilot = 5 seeds, 20 000 evaluations)

V1 — the encoding (H1). Gap to LB, mean over seeds:

Instance	DMO	QI-DMO (V1, no channel)	P-DMO linear twin (V1)	GA
n30 m5	2.7 %	2.0 %	2.1 %	0.8 %
n50 m10 bimodal	45.4 %	21.6 %	20.5 %	14.5 %
n100 m10	75.6 %	4.9 %	4.0 %	1.4 %
n50 m10 homogeneous	16.4 %	8.3 %	8.3 %	3.5 %

At n=100 the mean move size fell from 48.9 to 11.3 tasks and the population reached 51.9 s at 25 % of the budget (DMO: 101.9 s). The gain grows with n as predicted. But the Born-rule registers collapsed (end purity 0.996–0.999, end diversity 0.001–0.07) and 26–76 % of evaluations re-measured the parent's schedule.

V2 — decoherence dose-response (H2) and transfer (H1 on MRFO). Gap to LB for QI-DMO with $\gamma = c/n$:

Instance	c=0	c=0.25	c=0.5	c=1	c=2	wasted (c=0 → c≥0.25)
n30 m5	2.0 %	1.6 %	2.1 %	2.1 %	2.7 %	76 % → 1 %
n50 bimodal	21.6 %	15.9 %	15.9 %	18.5 %	21.3 %	14 % → 0 %

Instance	c=0	c=0.25	c=0.5	c=1	c=2	wasted (c=0 → c≥0.25)
n100 m10	4.9 %	3.9 %	5.5 %	8.1 %	11.2 %	26 % → 0 %
n50 homog	8.3 %	6.6 %	10.4 %	12.6 %	14.8 %	38 % → 0 %

QI-MRFO (mean makespan, gap in brackets):

Instance	MRFO	QI-MRFO c=0	QI-MRFO c=0.5	QI-MRFO c=1	GA	Max-Min
n30 m5	47.64 (7.0 %)	45.70 (2.6 %)	45.08 (1.2 %)	44.93 (0.9 %)	44.87 (0.8 %)	45.46
n50 bimodal	35.45 (41.2 %)	30.18 (20.2 %)	28.10 (11.9 %)	27.54 (9.7 %)	28.74 (14.5 %)	27.06
n100 m10	93.39 (98.5 %)	53.93 (14.6 %)	47.54 (1.0 %)	47.51 (1.0 %)	47.72 (1.4 %)	47.45
n50 homog	33.26 (17.2 %)	30.60 (7.8 %)	29.38 (3.5 %)	29.22 (2.9 %)	29.37 (3.5 %)	28.98

Effect sizes actually observed versus the HYPOTHETICAL prediction (20–40 % lower makespan at n=100): measured **49 %** lower makespan than MRFO (93.4 → 47.5) and **41 %** lower than DMO (82.6 → 48.9); the gap to the GA is not "below 5 %" but zero or negative (QI-MRFO ties or beats the GA on all four instances). The prediction was too conservative on the first part and right on the second.

V3 — mechanism tests. (a) DMO's unconditional next-position move explains QI-DMO's deficit: with a greedy version QI-DMO reaches 1.1 / 12.0 / 1.4 / 3.2 %, on par with QI-MRFO (classical DMO with the same change: 2.3 / 17.9 / 30.7 / 12.4 %). (b) Register attractor (alpha's superposition instead of its measured schedule): 7.1 / 34.2 / 87.8 / 14.2 % — purity stays at 0.1–0.2 and 24–90 tasks change per candidate: the search never concentrates. (c) Linear twin of QI-MRFO: 0.9 / 9.7 / — / 3.2 % versus Born 0.9 / 9.7 / 1.0 / 2.9 %: identical within seed noise. (d) Unsigned amplitudes: 0.9 % on n30 (= signed). (e) Population size 15/30/60: differences below one percentage point. (f) Runtime: QI-MRFO about 14 s versus MRFO about 3 s per 20 000 evaluations at n=100, m=10 in pure Python/NumPy (the objective is cheap; in a CloudSim-class simulator evaluation cost dominates and the overhead disappears).

What the mechanism changed (Phase 10). It changed one measurable search property, the distribution of move sizes (from about n/2 tasks per candidate to a purity-controlled few), which raised the improving-candidate fraction and the number of global-best improvements per 1 000 evaluations (n=100: 2.7 for QI-MRFO c=1 while MRFO improves on 1.4 % of candidates), and the decoherence channel changed a second property, the floor of that distribution (wasted fraction from 63 % to 5–17 %). Diversity is *not* what it improved: QI-MRFO's end diversity is about 0.001 (the population sits on the best schedule and explores by measurement noise), lower than MRFO's 0.02 and the GA's 0.09.

19. Ablation (Phase 11): did the quantum-inspired component cause the effect?

Ladder (n100 m10 unless noted)	gap to LB	verdict
MRFO (classical, floor encoding)	98.5 %	baseline failure
+ registers, Born measurement, basis-state attractor (QI-MRFO c=0)	14.6 %	the representation causes most of the gain
+ depolarising floor c=1 (QI-MRFO)	1.0 %	the floor causes the rest
Born rule replaced by linear mixing (P-MRFO twin, c=0.5; n50 bimodal / n30 / homog)	9.7 / 0.9 / 3.2 % versus 9.7–11.9 / 0.9–1.2 / 2.9–3.5 %	no measurable effect of the Born rule
signed → unsigned amplitudes (n30)	0.9 % versus 0.9 %	no measurable effect of interference

Ladder (n100 m10 unless noted)	gap to LB	verdict
basis-state attractor → superposition attractor (QI-DMO)	87.8 %	collapse-conditioned attraction is necessary
DMO's unconditional move → greedy (QI-DMO)	1.4 % (from 3.9 %)	host quirk, not mechanism

Answer: the effect is caused by (i) the superposition-and-measurement representation and (ii) the decoherence floor; the specifically quantum parts (amplitude squaring, sign/interference) contribute nothing measurable at this sample size. The mechanism is best described as a *measurement-based register representation with a purity-calibrated noise floor*; "quantum-inspired" is an accurate description of where it came from, not of why it works.

20. Failure analysis (Phase 12, adversarial; measured where stated)

- **Heavy-tailed static batches:** on the bimodal instance the Max-Min list heuristic (27.06) beats every metaheuristic (best QI-MRFO 27.54, GA 28.74). The landscape probe explains it: 43 % neutral 1-move neighbours and 38 % median barriers; a list heuristic that places big tasks first is structurally right. A population method must be seeded with Max-Min or given an objective a list heuristic cannot optimise (energy/cost/deadlines) to be worth its cost there.
- **Dynamic workloads (H4, the "big-result" hypothesis):** carrying the register state beats a restart on all four change types (AUC 2–4× lower) and beats the GA's carried population on drift, VM failure and VM addition; but the decoherence shock does **not** beat plain continuation at any churn severity between 10 % and 80 % (`lab_log.md`, severity table), and GA hypermutation — the classical analogue — is worse than GA continuation everywhere. At 80 % churn restart and continuation are indistinguishable. H4's strong form is falsified for uniform shocks; the boundary is measured.
- **Population collapse:** even with the floor, QI-MRFO's end diversity is about 0.001 and 5–47 % of candidates are wasted for small c ; the search relies on measurement noise around one schedule. This is a documented weakness, not hidden by the good makespans.
- **Overhead:** about 4–5× wall time and m × memory relative to the floor encoding in pure Python (measured); irrelevant in a simulator, relevant for embedded/edge use.
- **Adversarial cases from the notebook (fast mode: 3 seeds, 10 000 evaluations; `results/failure_cases_fast.csv`; 30-seed update in §26).**
- *17a tiny ($n=10, m=3$):* every method within 0.1–0.5 % of the lower bound, random search included (0.15 %); Max-Min is the worst (1.8 %). The mechanism is unnecessary here, as predicted.
- *17b two VMs ($n=40, m=2$):* all population methods identical (≤ 0.02 % gap); the floor encoding is lossless when $m = 2$, so the representational advantage vanishes, as predicted.
- *17c identical tasks on homogeneous VMs ($n=40, m=8$; optimum = 5 tasks per VM):* GA, Max-Min and QI-DMO reach the optimum (25.0) in every seed; **QI-MRFO reaches it in one seed of three** (28.3 ± 2.9), MRFO and DMO never (30.0). This landscape is a pure plateau: an improvement requires moving one task from a 6-task VM to a 4-task VM and every other move is neutral. QI-MRFO's strict greedy acceptance rejects neutral moves, and its collapsed population cannot drift; QI-DMO's unconditional next-position move — the very quirk that hurts it elsewhere — provides neutral drift and solves the instance. **A new, unpredicted boundary: when neutrality dominates, the decoherence floor is not a substitute for accepting neutral moves.** The obvious classical fix (accept equal-fitness candidates) was tested immediately (V4, 5 seeds, `lab_log.md`): it halves the plateau gap (16 % → 8 %) but does not reach the optimum in every seed, is neutral on the uniform instances and possibly harmful on the bimodal one (within one standard deviation). Greedy acceptance therefore stays the default; the plateau failure is a *population-collapse* problem (one point relocating on a plateau cannot find a two-step exchange quickly), which points to purity-regulated decoherence rather than to acceptance rules.

- *17d makespan + energy* ($n=100$, $m=10$, *additive objective*): QI-MRFO 0.212 = GA 0.212 $\approx$ Max-Min 0.211 (combined objective), QI-DMO 0.219, MRFO/DMO 0.37, random 0.40. The mechanism keeps its advantage over the classical hosts on the smooth objective, but so does the list heuristic: the energy term here is dominated by idle power during the makespan, so Max-Min remains hard to beat.
- *17e many VMs* ($n=40$, $m=20$): Max-Min 3.2 % $\ll$ QI-MRFO 12.7 % < GA 18.1 % < QI-DMO 33.7 % $\ll$ MRFO/DMO 80–91 %. With two tasks per VM the problem is heuristic-friendly and the population methods' 10 000 evaluations are far from enough; QI-MRFO still beats the GA. **Boundary: large m/n favours list heuristics.**
- *17f bimodal tall barriers* ($n=50$, $m=10$, 10 000 evaluations, $c=1$): QI-MRFO 26.41 (5.2 %) versus Max-Min 27.06 (7.8 %) and GA 29.25 (16.5 %) — the opposite of the 5-seed pilot at $c=0.5$ (28.10). The bimodal instance has the largest seed variance of the suite (sd 0.8–3.7); the 30-seed run decides this comparison (§26).

21. Version 2 improvement plan (what to change next, evidence-driven)

1. **Task-selective, severity-scaled decoherence** for dynamic environments: shock only the registers of tasks whose execution-time row changed, with γ proportional to the relative change (predicted to recover the churn/drift gains without harming structural changes) — the uniform shock is the proven-wrong version.
2. **Purity-regulated γ** (V3 of the static algorithm): set γ_t so that the population's expected move size stays in a target band (e.g. 1–3 tasks) instead of a fixed c/n ; prediction: removes the residual 5–47 % wasted evaluations at small c without the quality loss seen at large c .
3. **Max-Min seeding** (one individual initialised as $E(b_{MaxMin})$): predicted to close the heavy-tailed gap; must be tested against Max-Min + local search to avoid a trivial win.
4. **Objective where heuristics do not apply**: the energy-aware objective (Section 17d), then deadlines/cost, evaluated in CloudSim-class simulation.
5. **Drop the quantum-specific parts that do nothing** (sign, squaring) from the *default* configuration and keep the honest name: measurement-based register swarm with decoherence floor; keep the Born variant as an option with its documented convergence-speed difference.

22. Prior-art recheck after implementation (Phase 15;

`docs/literature/prior_art_recheck.md`)

Searched with our own terminology, the classical equivalents, quantum, cloud and optimisation terms. **(a) Already published:** the classical twin is essentially ICPSO (Strasser, Goodman, Sheppard & Butcher, GECCO 2016: one probability distribution per variable, unchanged PSO equations, renormalisation, one sample per evaluation, best-distributions shifted toward the sampled state — $\varepsilon = \mathbf{0}$ is exactly our basis-state attractor), with Pugh & Martinoli (2006) and Kennedy & Eberhart (1997) as precursors; Born-rule measurement of unit-vector registers is Han & Kim (2002) in the binary case and appears m-ary in QIEDA (Soloviev et al. 2021) and qudit imaginary-time evolution (Åsgrim & Awan 2025); a floor against full collapse exists as the H ε gate (Han & Kim 2004), EDA margins (Krejca & Witt 2018), Baluja's mutation shift and PBIL's pull-to-centre (Yang & Richter 2009); controlled forgetting at a change exists as PBIL hypermutation/hyper-learning and ACO pheromone equalisation after city deletion (Guntsch & Middendorf 2001). **(b) Separately published, never combined:** amplitude registers with swarm difference-vector updates; m-ary registers + floor + change-triggered shock. **(c) Not found:** a per-candidate depolarising channel in any EA/swarm; purity used to calibrate expected reassignments; a decoherence shock of strength γ_{shock} ; VM removal as projective column deletion with warm re-optimisation; any of this in a cloud scheduler. The defensible novelty is therefore narrow and must be stated as such: the purity-calibrated floor and its dose–response, the amplitude-versus-probability comparison

(negative), the structural-change rules and the failure-mode analysis of rounding encodings — positioned against ICPSO and QIEA, not as a new probabilistic-register swarm.

23. Possible research contribution (Phase 16/17)

In the requested form: *We identify the condition — task-to-VM scheduling with $n \gtrsim 50$ tasks under the floor encoding used by most swarm-scheduling papers — under which the classical optimizers DMO and MRFO exhibit the failure mode "move size $\approx n/2$, $\leq 2\%$ improving candidates, random-search-level makespan", and show that a measurement-based register representation with a purity-calibrated decoherence floor changes the property "expected number of reassigned tasks per candidate" (from $\approx n/2$ to a controllable few, with a measured dose–response in the floor strength) and therefore improves the outcome "makespan at equal evaluation budget" from a 76–98 % gap to $\approx 1\%$, matching a discrete GA. We further show (negative result) that the quantum-specific ingredients — amplitude squaring and signed interference — are measurably irrelevant, and that uniform decoherence shocks do not improve re-optimisation after workload changes at any tested severity, whereas the representation's structural adaptation rules do.*

Outcome type by the Phase 16 taxonomy: **B (works under a characterised regime: n large, heuristics inapplicable, warm re-optimisation) plus a clean negative for the "quantum" part and for uniform forgetting**. It is not outcome D (no advantage): the advantage over the algorithms the field actually uses (floor-encoded swarms) is large and reproducible (30 seeds, 7 instances, Holm-corrected $p < 10^{-4}$, Cliff's $\delta \approx -1$); the advantage over a proper discrete baseline is small or zero on static batches (GA-class; significantly better on one of seven instances) and real on dynamic ones; and a two-pass list heuristic beats it on six of seven static heterogeneous batches at this budget (§26). The contribution is therefore a *representation result with a measured boundary*, not a new best static scheduler.

24. Two-year extension (Phase 20)

- 0–3 months — literature + fundamentals: this report's maps; ICPSO/QIEA/EDA theory (genetic drift with r -valued variables, Ben Jedidia, Doerr & Krejca 2024); CloudSim 7G/iFogSim familiarisation.
- 3–6 months — algorithm + pilot: the notebook's full study on a held-out instance family and real traces (GoCJ, HPC2N, Google), Max-Min-seeded variants, energy/cost/deadline objectives; first workshop paper: "Why rounding breaks swarm schedulers and what a measurement-based representation fixes".
- 6–10 months — mechanism refinement: purity-regulated γ , task-selective severity-scaled shocks, structural rules for VM heterogeneity changes; dynamic benchmark suite with change generators; second paper (dynamic re-optimisation).
- 10–14 months — theoretical analysis: expected-move-size and drift analysis of the register swarm with a floor (runtime analysis in the style of Neumann's chance-constrained makespan work; margins/drift results from EDA theory transfer directly because the twin is a univariate model).
- 14–18 months — large-scale experiments: CloudSim/WorkflowSim with DAG workflows, 30+ seeds, DRL and list-scheduling baselines, statistical protocol as in Section 14 of the notebook.
- 18–21 months — prototype: a Kubernetes scheduler plugin/descheduler using the warm-started register swarm for pod placement under node churn (iContinuum/GreenK8s-style testbed).
- 21–24 months — publication + thesis.

Minimum viable thesis: the failure-mode analysis + representation fix + dose–response + dynamic structural rules, all reproducible (already in hand as a pilot). Main contribution: severity-aware re-optimisation with a measurement-based representation, with theory. Optional extension: Kubernetes prototype. Negative-result contribution (already obtained, publishable as a short paper): quantum-specific ingredients and uniform

decoherence shocks add nothing measurable — a caution to the growing "quantum-inspired scheduler" literature.

25. Exact next experiment

Task-selective decoherence under churn. Same harness as `observe_dyn_severity.py` (n=50, m=10, 15 000-evaluation warm start, 6 changes × 4 000 evaluations, 10 seeds), churn $\rho \in \{0.1, 0.2, 0.5\}$; strategies: `continue_struct` (control), uniform shock $\gamma \in \{0.25, 0.5\}$ (known-negative control), **selective shock** (registers of tasks sharing a VM with a replaced task receive γ_{sel}), and **severity-scaled shock** $\gamma_t = \kappa \cdot |ET'_t - ET_t|_1 / |ET_t|_1$ for drift. Metrics: post-change gap and AUC. Prediction (falsifiable): selective/scaled shocks reduce AUC below `continue_struct` by at least 10 % at $\rho = 0.5$ and are neutral at $\rho = 0.1$; if they do not, the forgetting line is closed and the programme concentrates on structural rules, purity regulation and the theory.

26. 30-seed confirmation (notebook `full` mode, executed in Docker; measured)

Part A — baseline (2 100 runs: 7 instances × 10 algorithms × 30 seeds; 20 000 evaluations each; `results/baseline_full.csv`, `results/stats_full.csv`). Three instance shapes are new relative to the pilot (held-out): n=200 m=20; n=100 m=10 lognormal tasks on low-heterogeneity VMs; n=60 m=8 bimodal tasks on homogeneous VMs.

Mean gap to the lower bound (%), 30 seeds:

Instance	Max-Min	Random	PSO	DMO	MRFO	GA	QI-MRFO	QI-DMO	P-MRFO (linear twin)
n30 m5 uniform, hetero	2.10	6.76	3.01	2.89	6.52	0.87	1.06	1.65	0.93
n50 m10 bimodal, hetero	7.77	34.50	14.06	40.85	42.15	13.55	11.99	18.04	13.27
n100 m10 uniform, hetero	0.83	95.34	33.84	75.39	83.94	1.44	1.12	3.96	1.25
n50 m10 uniform, homogeneous	2.10	15.23	6.44	16.05	17.70	3.55	3.14	7.17	3.17
n200 m20 uniform, hetero (held-out)	0.82	160.41	78.16	246.16	150.00	3.25	2.79	9.02	2.61
n100 m10 lognormal, low hetero (held-out)	0.79	14.82	4.40	15.79	13.67	1.07	1.07	3.16	0.92
n60 m8 bimodal, homogeneous (held-out)	3.96	19.83	11.65	18.46	21.64	8.04	8.68	9.23	10.34

Paired statistics (Wilcoxon signed-rank by seed, Holm-corrected across the 7 instances; Cliff's δ ; 95 % bootstrap CI of the mean difference in percentage points):

Comparison	Result over the 7 instances
QI-MRFO vs MRFO	QI-MRFO better on all 7: Holm $p < 10^{-4}$ everywhere, $\delta = -0.97 \dots -1.00$; differences -5.5 pp (n30) to -147 pp (n200)
QI-DMO vs DMO	QI-DMO better on all 7: Holm $p < 10^{-4}$, $\delta = -0.82 \dots -1.00$; -1.2 pp (n30) to -237 pp (n200)
QI-MRFO vs GA	equal or slightly better: significantly better only at n100 (-0.32 pp, CI [-0.54, -0.08], Holm $p = 0.011$, $\delta = -0.55$); raw $p < 0.05$ but not after Holm at n50-homogeneous (-0.41 pp) and n200 (-0.46 pp); no difference on the other four (
QI-MRFO vs P-MRFO (linear twin)	no significant difference on any instance after Holm (smallest Holm $p = 0.34$;

Comparison	Result over the 7 instances
QI-MRFO vs Max-Min	Max-Min significantly better on 6 of 7 (Holm $p \leq 0.013$; by 0.3 pp at n100 and lognormal, 1.0 pp homogeneous, 2.0 pp n200, 4.2 pp bimodal-hetero, 4.7 pp bimodal-homogeneous); QI-MRFO better only at n30 (−1.05 pp, Holm $p < 10^{-4}$)
QI-MRFO vs QI-DMO	QI-MRFO better on 6 of 7 (Holm $p \leq 0.002$), tie on bimodal-homogeneous
QI-DMO vs GA	GA better on 6 of 7 (Holm $p \leq 0.014$); tie on bimodal-homogeneous

Mechanism metrics (means over 30 seeds; largest instances): tasks changed per candidate — MRFO 126, DMO 121, QI-MRFO 19, linear twin 27, GA 7.6 (n200 m20); MRFO 46, QI-MRFO 10, GA 5.4 (n100). Global-best improvements per 1 000 evaluations at n200: QI-MRFO 5.1, GA 3.4, MRFO 0.48. Wasted candidates QI-MRFO 9–21 % ($c = 1$), end diversity 0.001–0.02, end purity 0.92–0.99. Runtime per 20 000 evaluations: QI-MRFO 2.5–4.6× MRFO (16–41 s vs 4–12 s in the Docker container with 18 concurrent processes); QI-DMO 2–3× DMO.

What the 30 seeds change relative to the pilot. (i) The encoding effect and its growth with n are confirmed with overwhelming evidence, including on the three held-out shapes ($n=200$, $m=20$: MRFO 150 % → QI-MRFO 2.8 %). (ii) The Born-rule irrelevance is confirmed: no instance separates QI-MRFO from its linear-probability twin. (iii) The pilot's "ties or beats the GA on all four instances" softens to "equal or slightly better, significant on one instance": QI-MRFO is a GA-class optimizer, not a better one. (iv) The pilot's suggestion that QI-MRFO matches Max-Min at $n=100$ (47.54 vs 47.45) does not survive: with 30 seeds Max-Min is significantly better on 6 of 7 static instances by 0.3–4.7 pp; QI-MRFO beats it only on the smallest instance. The boundary stated in §20 therefore widens: **on static independent-task makespan batches with heterogeneous VMs, a two-pass list heuristic is the right tool, and the population method's case rests on objectives and dynamics the heuristic does not handle** — which is exactly the direction the proposal takes.

Part B — ablation, sensitivity, adversarial and dynamic sections of the full run (measured; results/ablation_full.csv, sensitivity_full.csv, failure_cases_full.csv, dynamic_full.csv; the executed notebook is results/executed_full.ipynb, zero cell errors).

Ablation (30 seeds, 20 000 evaluations, 4 instances; mean gap to LB in %):

Variant	n30 m5	n50 bimodal	n100 m10	n50 homogeneous
MRFO (classical)	6.52	42.15	83.94	17.70
QI-MRFO no-decoherence (registers + measurement only)	2.46	17.74	13.12	7.84
QI-MRFO ($c = 1$)	1.06	11.99	1.12	3.14
QI-MRFO unsigned amplitudes	1.24	13.91	1.16	3.65
P-MRFO linear twin (no Born rule)	0.93	13.27	1.25	3.17
DMO (classical)	2.89	40.85	75.39	16.05
DMO greedy-next	2.26	17.65	30.03	12.58
QI-DMO no-decoherence	2.43	23.30	4.33	8.02
QI-DMO ($c = 0.25$)	1.65	18.04	3.96	7.17
QI-DMO greedy-next	1.33	10.12	1.46	3.69
P-DMO linear twin	1.71	16.25	4.72	7.53
QI-DMO register-attractor (negative control)	6.70	32.87	91.16	15.12
GA	0.87	13.55	1.44	3.55

The ladder holds at 30 seeds: representation first (84 % → 13 % at n100), decoherence floor second (13 % → 1.1 %); unsigned and linear-twin variants are within one standard deviation of QI-MRFO on every instance (SD

0.4–0.5 pp on the uniform instances, 7 pp on bimodal); the superposition attractor destroys the search; DMO's unconditional move explains the QI-DMO deficit (greedy-next QI-DMO: 1.46 % at n100, 10.1 % on bimodal — the best population result on that instance). One nuance the pilot missed: for QI-DMO the decoherence floor matters little (4.33 → 3.96 at n100) because DMO's own unconditional noise step already acts as a floor; for QI-MRFO, which is greedy in every phase, the floor is essential (13.1 → 1.1).

Sensitivity (30 seeds; n100 m10, n200 m20, n50 bimodal; gap %):

c in $y = c/n$	0	0.25	0.5	1	2	4
QI-MRFO n100	13.12	1.43	1.34	1.12	1.35	1.65
QI-MRFO n200 m20	75.04	5.64	3.36	2.79	3.37	5.05
QI-MRFO bimodal	17.74	12.32	12.19	11.99	11.78	11.32
QI-DMO n100	4.33	3.96	5.49	7.66	10.66	22.22
QI-DMO n200 m20	12.76	9.02	11.55	17.38	29.73	55.60
QI-DMO bimodal	23.30	18.04	16.63	18.17	22.22	25.48

QI-MRFO's wasted-candidate fraction falls monotonically with c (0.62 → 0.44 → 0.31 → 0.17 → 0.05 → 0.002 at n100) while quality is flat between c = 0.5 and 2: a broad, forgiving optimum. QI-DMO's optimum is sharp (c = 0.25) and large c is destructive (55.6 % at n200 for c = 4): the host's own noise compounds the channel. Population size 15/30/60 and somersault factor 1/2/3 change QI-MRFO by ≤ 0.4 pp at n100 (P = 15 is slightly better at n200: 2.42 vs 2.79 vs 3.22 for P = 60, i.e. more iterations help when the budget is tight).

Adversarial cases (30 seeds, 10 000 evaluations; gap % unless stated):

Case	Max-Min	Random	MRFO	QI-MRFO	GA	DMO	QI-DMO
17a tiny (n10 m3)	1.77	0.17	0.85	0.33	0.21	0.22	0.33
17b two VMs (n40 m2)	0.35	0.004	0.02	0.005	0.003	0.002	0.001
17c identical tasks, homogeneous VMs (n40 m8)	0.00	17.3	18.7	11.3 (27.83 ± 2.5)	0.00	17.3	3.3
17d makespan + energy (combined objective, lower is better)	0.211	0.400	0.375	0.212	0.213	0.375	0.218
17e many VMs (n40 m20)	3.19	96.6	96.6	15.4	18.6	90.7	33.2
17f bimodal tall barriers (n50 m10, 10k)	7.77	36.8	44.5	13.2	14.7	53.0	19.6

All four predicted boundaries are confirmed: tiny and two-VM instances need no mechanism (random search is within 0.2 %); the pure plateau defeats QI-MRFO (11.3 %) while the GA and Max-Min solve it exactly and QI-DMO nearly does (3.3 %, its unconditional move supplies neutral drift); many VMs per task and heavy-tailed batches belong to the list heuristic (Max-Min 3.2 % and 7.8 % versus QI-MRFO 15.4 % and 13.2 %); the fast-mode hint that QI-MRFO beats Max-Min on the bimodal instance was a 3-seed artefact. On the smooth makespan + energy objective QI-MRFO (0.212) equals the GA (0.213) and Max-Min (0.211) and beats its classical host (0.375) by 43 %.

Dynamic workloads (10 seeds; n50 m10; 15 000-evaluation warm start, then 5 changes × 4 000 evaluations; QI-MRFO c = 1; mean post-change gap % / area under the gap curve %):

Strategy	churn 20 %	drift	VM addition	VM failure
QI-MRFO restart	4.73 / 14.2	5.62 / 20.6	8.82 / 24.6	2.16 / 8.3
QI-MRFO continue (carried registers + structural rules)	4.12 / 5.6	4.40 / 8.8	6.96 / 8.8	2.13 / 3.2

Strategy	churn 20 %	drift	VM addition	VM failure
QI-MRFO shock $\gamma = 0.25$	4.23 / 8.2	4.27 / 9.0	7.73 / 12.5	2.10 / 3.8
QI-MRFO shock $\gamma = 0.5$	4.66 / 10.3	4.25 / 12.0	7.34 / 15.3	2.07 / 4.8
QI-MRFO shock $\gamma = 0.75$	4.75 / 13.4	5.03 / 17.7	8.18 / 19.8	2.20 / 6.2
MRFO restart / continue	51 / 50	79 / 72	76 / 72	34 / 15
GA restart	5.93 / 16.0	6.21 / 22.9	9.92 / 25.5	2.53 / 9.3
GA continue	5.00 / 7.2	5.51 / 10.0	8.92 / 11.5	2.81 / 5.1
GA hypermutation	5.79 / 11.2	6.37 / 15.9	10.50 / 14.2	2.82 / 8.6

Correction (V5 audit, risk R3): the notebook's dynamic runs used strategy `continue`. That strategy applies the VM-removal and VM-addition rules but does **not** reset the registers of replaced tasks under churn, which only `continue_struct` does. The churn column above is therefore plain continuation, and the label "structural rules" applies to the VM-failure and VM-addition columns only. The numbers are unaffected.

With 10 seeds the picture of the 5-seed pilot is unchanged and sharper: carrying the register state is the best or within noise of the best strategy on every change type, with recovery areas 2.5–3× smaller than a restart; **QI-MRFO-continue beats GA-continue on all four change types** (by 0.7–2.0 pp in final gap and 20–45 % in recovery area; seed SDs 0.4–1.1 pp); the decoherence shock never improves on continuation by more than 0.15 pp (drift, $\gamma \leq 0.5$) and always slows recovery; GA hypermutation is worse than GA continuation everywhere. The value of the mechanism for cloud resource management is therefore not "quantum forgetting" but the warm-startable, structurally adaptable state that a measurement-based representation provides.

27. V5 — critical exchange measurement (measured; development/held-out protocol)

27.1 Protocol upgrades (research quality)

V0–V4 had three weaknesses that the V5 protocol addresses:

- **Inference scope.** Every statistical test paired run seeds on a single instance per shape (\$26 uses `inst_seed = 1` for all 7 shapes), so its conclusions concern 7 instances, not 7 instance distributions.
- **Selection bias.** The decoherence constants were chosen on 4 of those 7 instances.
- **Loose bound.** On heavy-tailed batches the lower bound omits all but two terms of the $Q|pmtn|C_{max}$ bound.

V5 therefore uses: 1. a development set (TUNE = the 4 pilot instances), where parameters may be chosen and whose numbers are labelled selection-biased; 2. a held-out set (TEST = 8 new families × 10 new instance seeds 101–110 × 2 run seeds), where nothing is chosen; 3. the instance as the statistical unit; 4. the exact preemptive bound $C^{\{pmtn\}}_{\max} = \max \big(\max_{k \geq 2} \dots$; 5. write-once experiment directories (JSONL checkpoint, `jobs.json` digest, `meta.json` with the git commit, `DONE`); 6. a pre-registration committed before the run (`research_plan_v5.md`).

In addition, 212 unit tests including bit-exact golden fingerprints of V0–V4 guarantee that every V5 feature is opt-in. The unchanged smoke notebook re-executes to within 1.1e-16 of the committed results.

27.2 Observation (V5 OBSERVE)

Instrumenting QI-MRFO ($c = 1$) on the pilot instances (5 seeds, 20 000 evaluations; `results/v5_diagnostics.txt`, `results/v5_localopt.txt`) showed four things:

- **O1.** 31–46 % of evaluations re-evaluate a schedule already seen in the run. The earlier "wasted" metric (12–21 %) counted only parent duplicates.
- **O2.** Every improving candidate moves a task off the critical VM, which is a necessary condition for a strict makespan decrease, yet only 20–45 % of candidates do.
- **O3.** The global best stops improving at 16–54 % of the budget on three of four instances.
- **O4.** The end points are always relocation-optimal but leave 1–50 strictly improving critical swaps (50 at $n = 100$). Max-Min's schedule is relocation-optimal and never swap-optimal.

The binding constraint is therefore not the amount of noise, the thing adaptive decoherence would tune, but the *kind* of move the product-state measurement can produce. Tasks are resampled independently, so a correlated two-task exchange almost never happens.

27.3 Hypothesis H5 and mechanism

Critical exchange measurement (CXM). With probability p_x a measured candidate a additionally undergoes one exchange:

$$t \sim U\{i : a_i \in \arg \max_j \text{Load}_j(a)\}, \quad u \sim U\{i : a_i \neq a_t, L_i < L_t\}, \quad (a_t, a_u) \leftarrow (a_u, a_t).$$

If no shorter task exists, t is instead relocated to a uniformly random other VM. The measured pair then collapses: $\Psi_t \leftarrow \mathcal{D}_\gamma(E(a_t))$ and $\Psi_u \leftarrow \mathcal{D}_\gamma(E(a_u))$ (measurement back-action), so an accepted register encodes the exchange. Everything else is unchanged: the evaluation budget (one evaluation per candidate), the host dynamics, the decoherence floor and greedy acceptance.

Quantum reading: a correlated (non-product) measurement of a register pair. Classical equivalent: swap mutation restricted by two necessary conditions. The GA receives the identical operator (`run_ga(exchange=p_x)`) so the experiment can separate "generic problem knowledge" from "register-swarm-specific benefit".

Pre-registered predictions (`research_plan_v5.md` §7):

- **P1 (primary).** A lower held-out gap than unchanged QI-MRFO (pooled Wilcoxon over 80 instances, CI excluding 0).
- **P2.** A larger gain where more swaps were left unused.
- **P3.** End points closer to swap-optimal, later stagnation.
- **P4.** Linear twin still equivalent.
- **P5 (open).** How much the GA gains.

27.4 Development set (selection-biased)

On the 4 pilot instances (10 seeds, 400 runs; `results/h5_tune/`) the mean gap2 over the 4 instances fell from 3.73 % (QI-MRFO) to 0.82 % (QI-MRFO+CXM, $p_x = 1$), and from 4.98 % (GA) to 1.88 % (GA+CXM, $p_x = 1$). $p_x = 1$ and 0.5 tied on the pre-registered mean-rank criterion. The tie-break (lower mean development gap, giving 1.0) was fixed and committed before the held-out stage (`research_plan_v5.md` §9), and a $p_x = 0.5$ sensitivity arm was added to TEST. These numbers are optimistic by construction and are not used for any claim.

27.5 Held-out results (8 new families × 10 new instances × 2 seeds, 20 000 evaluations; results/h5_test/, results/h5_analysis.md)

Mean gap to the preemptive bound (%):

Family	Max-Min	GA	GA+CXM	P-MRFO	P-MRFO+CXM	QI-MRFO	QI-MRFO+CXM
n80 m8 uniform high	1.009	1.232	0.922	0.989	0.043	1.046	0.061
n150 m15 uniform high	0.955	2.157	2.158	1.721	0.063	1.790	0.134
n300 m30 uniform high	0.815	5.819	5.163	9.123	0.169	9.075	0.329
n100 m10 bimodal high	0.267	3.225	0.844	4.008	0.039	3.660	0.144
n200 m10 bimodal none	0.137	0.389	0.269	0.347	0.004	0.527	0.006
n120 m12 lognormal high	0.286	1.563	1.536	1.052	0.045	1.300	0.075
n60 m12 uniform low	1.609	4.039	3.226	3.611	0.208	4.146	0.274
n100 m20 lognormal low	0.000	4.457	2.251	5.855	3.458	5.628	3.307

Mean rank over the 160 (instance, seed) blocks: P-MRFO+CXM 1.57, QI-MRFO+CXM 2.16, Max-Min 3.20, GA+CXM 5.01, P-MRFO 5.38, GA 5.47, QI-MRFO 5.49, Min-Min 8.18, MRFO 8.55.

Paired comparisons. Unit = instance (mean of 2 run seeds); difference in gap2 percentage points; the pooled test is over 80 instances; Holm correction across the 8 families:

Comparison	Pooled difference [95 % CI]	Wins / ties / losses	Pooled p	Families (Holm p < 0.05)
P1: QI-MRFO+CXM – QI-MRFO	–2.86 [–3.58, –2.21]	76 / 0 / 4	< 1e-4	7 of 8 better (10/10 wins each); lognormal-low 6/4, n.s.
sensitivity p_x = 0.5 – QI-MRFO	–2.81 [–3.49, –2.20]	78 / 1 / 1	< 1e-4	8 of 8 better
P4: P-MRFO+CXM – QI-MRFO+CXM	–0.04 [–0.16, +0.10]	68 / 1 / 11	< 1e-4	4 of 8 favour the linear twin , none the Born rule
P5: GA+CXM – GA	–0.81 [–1.21, –0.46]	50 / 2 / 28	1e-4	none
QI-MRFO+CXM – GA+CXM	–1.50 [–1.94, –1.08]	73 / 1 / 6	< 1e-4	7 of 8 better; lognormal-low n.s. (GA+CXM ahead)
QI-MRFO+CXM – Max-Min	–0.09 [–0.45, +0.33]	67 / 2 / 11	< 1e-4	6 of 8 better; bimodal-high n.s. (8/2); lognormal-low worse (0 wins, 2 ties, 8 losses)
reference: QI-MRFO – Max-Min	+2.76 [+2.08, +3.51]	7 / 0 / 73	< 1e-4	Max-Min better on 6 of 8 (n200 bimodal-none Holm p = 0.055; n80 5/5)
reference: QI-MRFO – P-MRFO (no CXM)	+0.06 [–0.29, +0.42]	38 / 1 / 41	0.64	none

The pooled mean against Max-Min has a CI that includes zero only because of the lognormal-low family, where the difference is +3.3 pp. The median and the win rate strongly favour QI-MRFO+CXM.

Mechanism diagnostics (means over TEST runs, QI-MRFO → QI-MRFO+CXM):

- improving critical swaps left at the end point 139.7 → 10.9;
- last global-best improvement at 54 % → 76 % of the budget;
- late-half improving fraction 1.45 % → 0.44 %;
- global duplicate evaluations 27.9 % → 2.5 %, parent-identical 12.5 % → 0.01 %;
- candidates touching the critical VM 33 % → 85 %;
- end diversity 0.007 → 0.112, end purity 0.974 → 0.966;
- move size 14.0 → 24.3 tasks;

- runtime 2.9 → 3.9 s per run.

Convergence (pooled gap2 at 5 / 10 / 25 / 50 / 100 % of the budget) is 9.2 / 3.8 / 1.3 / 0.74 / 0.54 % with CXM against 38.2 / 25.6 / 12.4 / 6.0 / 3.4 % without.

27.6 Verdicts on the pre-registered predictions

- **P1 confirmed (primary) → CXM retained** as the recommended QI-MRFO option for makespan. The effect is large (rank-biserial -0.96), holds on 7 of 8 families at 10/10 instances each, and survives the tie-break sensitivity arm.
- **P3 partially supported.** The end points are much closer to swap-optimal and stagnation comes later, but the late-half improving fraction fell rather than rose. CXM reaches near-optimal schedules so early that little is left to improve in the second half.
- **P2 not supported as pre-registered.** The absolute effect is capped by each family's baseline gap. A post hoc relative version (`results/h5_posthoc.md`, exploratory) does follow the mechanism: 92–98 % of the gap is removed on 7 families and 22 % on the family with almost no unused swaps; $p = 0.71$ across families, 0.35 across instances.
- **P4 falsified.** Under CXM the Born rule is measurably *worse* than the classical linear-probability twin. The margins are small (0.002–0.16 pp) but consistent (68/80 instances). This is the first measurement in the project in which the quantum-specific ingredient matters at all, and it matters in the wrong direction.
- **P5 answered.** The exchange operator alone is not what matters: the GA gains only a third as much and is beaten by QI-MRFO+CXM on 73/80 instances. The gain comes from the interaction between CXM and the register swarm: greedy acceptance around a collapsed best, plus back-action that stores the exchange in the accepted register. Together they make CXM a stochastic swap-descent around the best-known schedule.

27.7 New boundary: the big-task plateau

QI-MRFO+CXM beats Max-Min on 6 of 8 held-out families, which reverses §26's "Max-Min wins 6 of 7" for the unchanged algorithm. It loses on n100 m20 lognormal/low-heterogeneity. On all 10 instances there the preemptive bound equals $L_{\max}/S_{\max}$, meaning the largest task runs alone on the fastest VM, and Max-Min attains it exactly, so it is provably optimal (`results/h5_posthoc.md`). The swarm reaches that schedule on only 2 of 10 instances. Moving the largest task to the fastest VM pays off only after that VM has been emptied, and every emptying move is makespan-neutral and rejected by strict acceptance. This is the same plateau mechanism as the V4 identical-task case (§20), now on realistic heavy-tailed instances. Candidate remedies are heuristic seeding and plateau-aware acceptance with a lexicographic tie-breaker (UNMEASURED).

27.8 What changes in the claims

- **Static makespan batches.** With CXM the population method is no longer "GA-class and beaten by Max-Min". On the held-out families it is the best method tested except where the optimum is a constructive corner case that Max-Min reaches exactly. (*Qualified by §29: a (1+1)-EA with the same moves is at least as good, so the gain belongs to CXM, not to the swarm.*)
- **Quantum-specific ingredients.** They remain unhelpful. The recommended configuration is the classical twin (probability vectors, linear mixing) with the decoherence floor and CXM. "Quantum-inspired" describes the derivation (registers, measurement, back-action, correlated two-register measurement), not a source of advantage.
- **Novelty (positioned against §22).** The correlated two-register exchange with measurement back-action inside a register swarm is not in the prior-art recheck. Its classical ingredient (swap mutation) is textbook. The contribution is the measured finding that the product-state measurement cannot express the move a

relocation-optimal schedule needs, and that adding it gives a larger gain to the register swarm than to a GA.

28. V5 — re-optimisation under change with migration cost (H6; measured)

Question. Section 26 showed that carried registers beat restart and GA continuation, but it counted only makespan. A running cloud also pays for every task the new schedule *migrates*. The H6 experiment asks three things: 1. Does CXM's static gain survive under change? 2. Does carrying the repaired previous best schedule (fixing audit risk R4) speed recovery? 3. How does the swarm compare with the obvious practitioner baseline, recomputing Max-Min after every change, once migrations are counted?

It was pre-registered in `research_plan_v5.md` §11. **Design:** six held-out scenarios, 10 seeds (instance seeds 101–110), $K = 8$ changes, a 20 000-evaluation warm start and 4 000 evaluations per epoch, 600 runs (`results/h6_dynamic/`, `results/h6_analysis.md`). Voluntary migrations count persistent tasks whose VM survived but that the new deployed schedule moves.

Strategy (pooled over 60 scenario-seed pairs)	Post-change gap (%)	Recovery AUC (%)	Voluntary migrations per epoch
Max-Min recomputed every epoch	0.80	0.80	87.5
Incremental list heuristic (no voluntary migration)	77.0 (1.3–239 by scenario)	same	0
GA continue / GA+CXM continue	4.08 / 4.48	7.44 / 6.57	20.6 / 40.0
QI-MRFO continue / + elite	3.38 / 3.03	6.88 / 6.14	19.4 / 17.0
QI-MRFO+CXM continue / + elite	0.41 / 0.47	1.86 / 1.80	58.3 / 51.6
P-MRFO+CXM continue + elite (classical twin)	0.29	1.58	56.1
QI-MRFO+CXM restart	0.85	12.84	93.9

Verdicts:

- **H6a (primary) confirmed.** CXM under change: -2.98 pp [-3.79 , -2.30], better on 60/60 pairs; all 6 scenarios are Holm-significant.
- **H6b rejected as pre-registered.** Elite carry-over: AUC -0.06 pp, CI [-0.19 , $+0.08$]. It helps after churn and VM failure (Holm $p = 0.012$) but **hurts after VM addition**: post-change gap $+0.44$ pp, 0/10, Holm $p = 0.012$. The carried schedule leaves the new VM empty, becomes the global-best attractor, and an exchange cannot move a task onto an empty VM.
- **H6c.** The register swarm beats the GA (both with CXM) on 60/60, by 4.0 pp. CXM makes the GA worse under change ($+0.40$ pp, 16/44). Carried state lowers the recovery AUC by 11 pp relative to a restart (60/60).
- **H6d confirmed (pooled).** QI-MRFO+CXM+elite Pareto-dominates recomputing Max-Min: a lower post-change gap (-0.33 pp, 48/60, $p = 1e-4$) and 41 % fewer voluntary migrations (60/60). The exception is n200 m20 lognormal mixed, where Max-Min's gap is lower (0.25 % vs 0.82 %).
- **Replication.** The classical linear twin again beats the Born rule (-0.17 pp, 55/60).

Lesson. CXM roughly triples the swarm's migrations (19 → 58 per epoch at $n = 100$), because the objective gives no reason to leave tasks in place. The pooled Pareto dominance over Max-Min holds, but half of the persistent tasks still move every epoch. Migration cost has to enter the objective: a problem that list heuristics cannot address and that a carried-state population method can (next hypothesis).

29. V5 — is the register swarm needed once CXM exists? Max-Min seeding (H7; measured)

Why. Late in a QI-MRFO+CXM run every candidate is "best schedule + per-task noise at rate c/n + critical exchange", which is exactly the mutation of a (1+1)-EA. Without that minimal control, H5 could not say whether the register swarm contributes anything beyond the exchange move. H7 (pre-registered, `research_plan_v5.md` §12) adds it, tuned with comparable effort on the development set, and tests Max-Min seeding. It runs on a **fresh** held-out set (instance seeds 201–210), so the seeding remedy is not evaluated on the instances that exposed the plateau failure (`results/h7_test/`, `results/h7_analysis.md`, 1 200 runs).

Comparison (pooled over 80 fresh instances; gap2 pp)	Difference [95 % CI]	Wins A / ties / wins B	p
H7a: QI-MRFO+CXM – (1+1)-EA+CXM	+0.18 [–0.11, +0.53]	21 / 1 / 58	1e-4 (favours the (1+1)-EA)
P-MRFO+CXM – (1+1)-EA+CXM	–0.001 [–0.21, +0.23]	26 / 0 / 54	0.12
H7b: QI-MRFO+CXM+seed – QI-MRFO+CXM	–0.51 [–0.88, –0.22]	67 / 2 / 11	< 1e-4
H7c: QI-MRFO+CXM+seed – (1+1)-EA+CXM+seed	+0.014 [+0.004, +0.025]	19 / 10 / 51	0.001
QI-MRFO+CXM+seed – Max-Min	–0.58 [–0.72, –0.45]	70 / 10 / 0	< 1e-4
P-MRFO+CXM+seed – QI-MRFO+CXM+seed (P4 replication)	–0.015 [–0.022, –0.008]	54 / 10 / 16	< 1e-4
QI-MRFO+CXM – Max-Min (H5 replication)	–0.07 [–0.41, +0.35]	63 / 2 / 15	3e-4

Mean rank over the 160 blocks: (1+1)-EA+CXM+seed 2.70, P-MRFO+CXM+seed 3.10, (1+1)-EA+CXM 3.39, QI-MRFO+CXM+seed 3.85, P-MRFO+CXM 3.98, QI-MRFO+CXM 5.31, GA+CXM+seed 6.40, Max-Min 7.27.

Verdicts.

- **H7a (primary) failed.** A (1+1)-EA with the same moves beats QI-MRFO+CXM on 58 of 80 fresh instances. It is Holm-significantly better on the two largest uniform families, and no family favours the swarm. By the rule committed before the run, **the register swarm is unnecessary for static makespan once the exchange measurement exists**, and H5's static gain belongs to CXM, not to the swarm.
- **Why.** The (1+1)-EA wastes 36 % of its evaluations on duplicates and moves only 1.9 tasks per candidate, yet it converges faster: 5.7 % gap at 5 % of the budget against 10.0 % for the swarm. Splitting the budget across 30 individuals costs more than the MRFO dynamics return at 20 000 evaluations.
- **H7b confirmed.** Seeding with the Max-Min schedule improves QI-MRFO+CXM on 67/80 instances and removes the big-task plateau failure of §27.7 (3.11 % → 0.02 %). It worsens no family. The seeded swarm is never worse than Max-Min and is strictly better on 70/80 instances.
- **H7c.** The seeded (1+1)-EA, i.e. "Max-Min + stochastic CXM local search", the control §21 demanded, is marginally better than the seeded swarm (51 vs 19 wins, +0.014 pp).
- **Quantum-specific part.** Replicated again: the linear twin is better than the Born rule (54/16).

Consequence for the claims of §27.8. The static makespan result must be restated: the decisive V5 ingredient is the correlated exchange measurement, and the best static method tested is **Max-Min seeding + a (1+1)-EA with the CXM moves**. The register swarm's remaining case rests on re-optimisation under change: carried registers and structural rules (§26, §28). H8 (§30) tests that against a (1+1)-EA carrying its single schedule, an arm fixed before the H7 result was known.

30. V5 — migration-aware re-optimisation (H8; measured)

Why. §28 showed that CXM roughly triples migrations because the makespan objective ignores them. H8 (pre-registered, `research_plan_v5.md` §14 and amendment §14.1) prices them. After the first epoch every method optimises $\text{cost}(a) = C_{\max}(a) (1 + \lambda \text{mig}(a)/\text{eligible})$, relative to the previous *deployed* schedule. Six scenario types, fresh seeds 201–210, $K = 8$, $\lambda \in \{0.05, 0.2, 1.0\}$; 1 260 runs (`results/h8_migration/`, `results/h8_analysis.md`). The strongest heuristic is a **Chooser**: each epoch it deploys whichever of "recompute Max-Min" and "keep everything, place only new or orphaned tasks" is cheaper under the true cost.

Pooled mean cost gap (%):

Strategy	$\lambda = 0.05$	$\lambda = 0.2$	$\lambda = 1.0$
Max-Min recomputed / Incremental / Chooser	4.69 / 77.76 / 3.40	16.63 / 77.76 / 8.96	80.31 / 77.76 / 29.99
GA continue	5.89	8.71	19.30
(1+1)-EA+CXM continue	9.31	11.61	19.36
QI-MRFO+CXM continue	3.19	7.48	19.48
P-MRFO+CXM continue	3.28	7.43	19.72

Verdicts.

- **H8a (swarm < Chooser).** Retained at $\lambda = 0.2$ (−1.48 pp [−2.62, −0.30], Holm $p = 0.048$) and $\lambda = 1.0$ (−10.5 pp [−15.6, −5.5], Holm $p = 0.005$). Not retained at $\lambda = 0.05$: the CI [−0.84, +0.81] includes 0 despite 39/21 wins.
- **H8b (swarm < GA).** Retained only at $\lambda = 0.05$ (48/12). At higher λ the GA, which migrates least, wins more pairs, though not significantly.
- **(1+1)-EA arm.** No significant difference at any λ . The (1+1)-EA wins more pairs but is catastrophic after VM additions.
- **Linear twin vs Born rule.** No difference.

Mechanism: heterogeneity by change type.

- **Coordinated reconfiguration (drift, mixed events, VM addition).** The swarm wins against the Chooser at every λ (Holm $p \leq 0.016$ at $\lambda = 1.0$).
- **Local change (churn, VM failure).** It loses (Holm $p = 0.012$ at $\lambda \geq 0.2$), because zero-migration repair is already near-optimal there. The swarm's population is sampled *from* the carried registers and never contains the exact deployed schedule, so it pays migrations it does not need.
- **VM addition (post hoc, exploratory).** On a balanced deployed schedule every single-task move onto the new VM is uphill under the price (30/30 cases). The (1+1)-EA therefore never moves: 40.8 % cost gap, zero migrations. The swarm's VM-addition rule (a uniform share of amplitude for the new VM in every register) produces multi-task candidates that do escape. The exact structure that makes them downhill is not isolated; a naive coordinated move is downhill in only 20 % of cases.

Consequence. This is the first setting in V5 where the register representation does something a (1+1)-EA with the same moves cannot. Its structural rule for new capacity creates coordinated moves across a barrier that the migration price builds for local search. The data also say what to fix: anchor the swarm on the deployed schedule (an elite) for local changes (H10, pre-registered next).

31. V5 — the deployed schedule as an elite anchor under migration pricing (H10; measured)

Why. In H8 the swarm lost to zero-migration repair after churn and VM failure. Its population is *sampled from* the carried registers and never contains the exact deployed schedule, which costs nothing to keep. H10 (pre-registered, `research_plan_v5.md` §17; fresh seeds 301–310; 900 runs; `results/h10_analysis.md`) adds that schedule as an elite.

Pooled cost gap % (migrations per epoch)	$\lambda = 0.05$	$\lambda = 0.2$	$\lambda = 1.0$
Chooser	3.41 (49.3)	8.72 (31.6)	30.19 (11.3)
GA continue	5.02 (15.6)	8.33 (11.5)	18.95 (4.3)
(1+1)-EA+CXM continue	9.36 (27.6)	11.86 (18.9)	19.21 (7.6)
QI-MRFO+CXM continue	2.79 (42.2)	7.35 (28.2)	19.64 (12.1)
QI-MRFO+CXM continue + elite	4.21 (30.2)	9.79 (18.9)	19.35 (7.6)

Verdicts.

- **H10a (primary) rejected at every λ .** Elite – no elite: +1.43, +2.44 and –0.29 pp, none significant. The elite wins most pairs at $\lambda \geq 0.2$ (42/18) but loses by large margins on one change type:
- after churn and VM failure it wins 10/10 at every λ (Holm $p = 0.012$) and cuts migrations sharply;
- **after VM addition it loses** by 9–19 pp with 0–1/10 wins, and its VM-addition migrations drop to zero at $\lambda = 1$. This is the same mechanism as H6b, now seen a second time under a different objective. The carried schedule leaves the new VM empty and, as the global-best attractor, suppresses the coordinated escape of §30.
- **H10b.** The elite-anchored swarm beats the Chooser only at $\lambda = 1.0$ (–10.8 pp).
- **Replication of H8a on fresh seeds.** The swarm without elite beats the Chooser at $\lambda = 0.05$ (43/17, Holm $p = 1e-4$) and $\lambda = 1.0$. At $\lambda = 0.2$ the CI excludes 0 but Wilcoxon $p = 0.074$.
- **Figure.** `results/fig_v5_migration_heatmap.png` shows the contrasts by scenario type and λ .

Consequence. The data point to an event-aware rule, derived post hoc and therefore tested separately on fresh seeds as H11 (§33): carry the elite after every event except a VM addition.

32. V5 — purity-regulated decoherence, the §21 item-2 prediction (H9; measured)

Why. The brief puts feedback-controlled decoherence first. §21 of this report, written before V5, had predicted that holding the expected move size $n(1 - \bar{\pi})$ in a 1–3-task band "removes the residual wasted evaluations without quality loss". H9 (pre-registered, `research_plan_v5.md` §18) tested exactly that controller, untuned, with and without CXM, on fresh static instances (seeds 301–310; 640 runs; `results/h9_analysis.md`).

Pooled over 80 instances	QI-MRFO $\rightarrow$ + band	QI-MRFO+CXM $\rightarrow$ + band
global duplicate evaluations	27.4 % $\rightarrow$ 46.4 % (worse on 78/80)	2.5 % $\rightarrow$ 9.4 % (worse on 77/80)
gap2	3.39 % $\rightarrow$ 4.12 % (+0.72 pp [0.20, 1.34], $p = 0.003$)	0.34 % $\rightarrow$ 0.49 % (+0.15 pp [0.04, 0.29]; 52/26 wins, $p = 0.12$)

Verdict: falsified in both settings. The controller increases the waste it was meant to remove. Without CXM the gap is also significantly worse; under CXM it breaks the +0.10 pp non-inferiority margin.

Mechanism. The controller lowered γ (mean c 0.33–0.64 on 7/8 families without CXM, 0.06–0.82 with CXM), because $n(1 - \bar{\pi})$ exceeded 3 tasks. That quantity is correct in expectation but is dominated by a few diffuse

registers, while duplicates come from the many collapsed ones. Lowering γ therefore starves precisely the registers that produce duplicates, reproducing V2's low-c waste. The move-size distribution is bimodal (many zero moves plus some large ones), so mean purity is a poor control signal. A controller on a direct signal, such as the per-register or parent-identical duplicate rate, is UNMEASURED.

33. V5 — event-aware elite under migration pricing (H11; measured, fresh seeds)

Why. Twice (H6b, H10a) the carried elite helped after every event type except a VM addition. After a VM addition the carried schedule leaves the new VM empty and, as the attractor, blocks the register swarm's coordinated escape (§30). The rule "no elite after a VM addition" was derived post hoc, so it was pre-registered (research_plan_v5.md §21) and tested on **fresh seeds 401–410** (720 runs; results/h11_analysis.md).

Pooled cost gap %	$\lambda = 0.05$	$\lambda = 0.2$	$\lambda = 1.0$
Chooser (best heuristic)	3.44	8.72	29.78
QI-MRFO+CXM continue, no elite	3.02	7.23	19.26
... + elite always	4.37	9.90	19.59
... + event-aware elite	2.61	6.45	17.07

Verdicts.

- **H11a retained at every λ .** Event-aware – no elite: -0.41 , -0.78 and -2.19 pp; 41/9, 41/9 and 40/10 wins; Holm $p < 1e-4$.
- **H11b retained at every λ .** Event-aware – always-elite: -1.76 , -3.45 and -2.52 pp.
- **H11c.** The event-aware swarm beats the Chooser at **every** λ : -0.83 , -2.27 and -12.7 pp. It wins on drift, mixed events, VM failure and (at $\lambda \geq 0.2$) VM addition. It is still worse on pure churn at $\lambda \geq 0.2$, where zero-migration incremental repair is best.
- **Replication.** The unconditional elite remains n.s. on fresh seeds.

Reading. The rule combines two mechanisms, each where it helps: the deployed schedule as a zero-migration anchor after local events, and the register representation's uniform-share rule for new capacity, which produces coordinated multi-task moves, after a VM addition. Of all V5 results this is the most specific to the register representation. The (1+1)-EA with the same moves cannot make these moves (§30), and this configuration beats the best heuristic chooser at every migration price tested. The remaining boundary (churn) suggests repairing the elite by greedy placement of new tasks; that is UNMEASURED.

34. V5 summary — every hypothesis, its test, its verdict

All hypotheses were pre-registered in research_plan_v5.md before their runs, and every result directory is write-once with its git commit in meta.json. Unit = instance (static) or scenario-seed pair (dynamic). Held-out seeds were never used for any choice. Figures: results/fig_v5_convergence.png, results/fig_v5_dynamic_tradeoff.png, results/fig_v5_migration_heatmap.png, results/fig_v5_h12_heatmap.png, results/fig_v5_h13_heatmap.png.

#	Hypothesis	Held-out test	Verdict
H5	Critical exchange measurement (CXM) lowers the static gap	80 instances (seeds 101–110)	confirmed (-2.86 pp, 76/80); beats Max-Min on 6/8 families; Born rule worse than the linear twin; GA gains far less

#	Hypothesis	Held-out test	Verdict
H6	CXM under change; elite carry-over; vs Max-Min recompute with migrations	60 pairs (seeds 101–110)	H6a confirmed (60/60); H6b rejected (hurts after VM addition); H6d confirmed (lower gap, 41 % fewer migrations than Max-Min recompute)
H7	The swarm is needed once CXM exists; Max-Min seeding	80 fresh instances (seeds 201–210)	H7a failed : a (1+1)-EA with the same moves wins 58/80, so the static gain belongs to CXM; H7b confirmed (seeding fixes the plateau family)
H8	Migration-priced re-optimisation beats the best heuristic chooser	60 pairs $\times$ 3 λ (seeds 201–210)	retained at $\lambda = 0.2$ and 1.0 ; heterogeneous by change type; the (1+1)-EA is stuck after VM additions
H9	Purity-regulated γ (report §21 item 2) removes waste without loss	80 instances (seeds 301–310)	falsified with and without CXM (duplicates rise, gap worsens)
H10	The deployed schedule as an elite anchor under migration pricing	60 pairs $\times$ 3 λ (seeds 301–310)	rejected pooled (wins 10/10 on churn and VM failure, loses 0–1/10 on VM addition)
H11	Event-aware elite (none after VM addition)	60 pairs $\times$ 3 λ (seeds 401–410)	confirmed at every λ ; beats the best heuristic chooser at every λ ; remaining boundary: pure churn at $\lambda \geq 0.2$
H12	Incremental elite (new churn tasks placed by list scheduling)	60 pairs $\times$ 3 λ (seeds 501–510)	retained at $\lambda = 0.2$ and 1.0 , not at $\lambda = 0.05$ (26/3, but the CI of the mean includes 0); removes the churn boundary at $\lambda \leq 0.2$; vs a (1+1)-EA with the same repair the swarm wins only through VM additions
H13	Event-aware decoherence (no floor after local changes)	60 pairs $\times$ 3 λ (seeds 601–610)	retained at $\lambda = 0.2$ and 1.0 , not at 0.05 (36/14, CI includes 0); the VM-addition exception is not needed (H13b X); the final configuration beats the Chooser at every λ (58/2, 57/3, 59/1)
H14	The swap move (CXM) transferred to QI-DMO, all phases, $p_x = 1$	80 instances (seeds 701–710)	rejected : worse on 65/80 (+2.33 pp), 6/8 families Holm-worse; descriptively better at every scaling size (500–5000 tasks)
H15	CXM only in QI-DMO's greedy phases, $p_x = 0.1$ (tuned on the development set)	80 fresh instances (seeds 801–810)	rejected narrowly : -0.81 pp [$-1.43, -0.25$], 47/80, but Wilcoxon $p = 0.067$; no family worse; beats the H14 version on 71/80

Threats to validity (V5)

- **Synthetic workloads only.** All instances come from the project's generator: independent tasks, uniform / lognormal / bimodal lengths, three VM-heterogeneity levels. Real traces (GoCJ, Google, Alibaba) and DAG workflows are UNMEASURED.
- **Fixed budgets and sizes.** Static runs use 20 000 evaluations; dynamic runs use $20\,000 + 8 \times 4\,000$ per scenario, $P = 30$, $n \leq 300$. Other budgets, and whether the population pays off at larger budgets (H7 alternative (a)), are UNMEASURED.
- **Tuning asymmetry.** p_x was tuned for QI-MRFO+CXM and the GA, and (c , p_x) for the (1+1)-EA, but $c = 1$ was not re-tuned for the swarm under CXM. This favours the control, not the swarm, so it cannot explain H7a in the swarm's favour.
- **Migration cost model.** Multiplicative, one λ per run, every migration equal. Real costs depend on task state size and network (UNMEASURED); three λ values bracket the trade-off.
- **Multiplicity across hypotheses.** Each hypothesis has its own pre-registered primary test; there is no correction across H5–H11. Within a hypothesis Holm is applied across families or λ .
- **Post hoc elements.** P2's relative re-expression, the H8 barrier check and the H11 rule are post hoc. They are labelled, and the one that became a claim (H11) was re-tested on fresh seeds. H12 came from a development-seed observation and was pre-registered before its fresh-seed run. Its λ -conditional recommendation is a post hoc reading of a pre-registered rule that was only partly met. H13's rule came from development seeds as well; its λ -conditional decision rule was pre-registered.
- **Same generator shapes.** H7 reuses H5's family shapes with new seeds, and the dynamic scenarios reuse shapes across H6/H8/H10/H11/H12/H13 with disjoint seeds. Generalisation beyond these shapes is UNMEASURED.

Status of the §21 improvement plan and the §25 "exact next experiment" after V5

§21 / §25 item	V5 status
1. Task-selective, severity-scaled decoherence under change (§25)	Not run: superseded. CXM under change (H6a, 60/60) and the event-aware elite (H11) addressed recovery. Selective shocks remain UNMEASURED.
2. Purity-regulated γ	Tested (H9) and falsified for the band controller as specified. Under CXM the fixed floor no longer affects the static gap for $c \in [0, 2]$ (§35, descriptive), so little headroom is left for any γ controller there. Under migration pricing an event-conditioned floor works (H13, §37) : no floor after local changes, retained at $\lambda \geq 0.2$. A controller on a direct duplicate signal is UNMEASURED.
3. Max-Min seeding	Tested (H7b), confirmed . Against Max-Min + local search (H7c) the seeded (1+1)-EA is marginally better.
4. Objectives where heuristics do not apply	Migration-priced re-optimisation tested (H8, H10–H13) . Energy, SLA and monetary cost are UNMEASURED in V5.
5. Drop the quantum-specific parts from the default	Supported by data . The linear twin beats the Born rule under CXM (H5: 68/80; H6: 55/60; H7: 54/16), and is n.s. under migration pricing (H8).

35. V5 — the decoherence floor once CXM exists (descriptive, development set)

Question. Does the fixed floor $\gamma = c/n$ still matter once CXM exists? If not, an adaptive γ has nothing to adapt.

Design. 4 development instances, 10 seeds, 20 000 evaluations. $c \in \{0, 0.25, 0.5, 1, 2, 4\}$ with CXM, and a control committed before it ran: $c = 0$ vs $c = 1$ without CXM. 640 runs; run-level pairs, descriptive.
(`results/v5_c_under_cxm.md`, `results/v5_c_without_cxm.md`, `results/v5_c_sweep_paired.md`)

Mean gap2 %, $c = 0 \rightarrow c = 1$	QI-MRFO (Born)	P-MRFO (linear twin)
without CXM	9.85 $\rightarrow$ 3.73 ($c = 1$ better on 38/40 runs)	19.68 $\rightarrow$ 4.56 (38/40)
with CXM	0.63 $\rightarrow$ 0.82 (−0.19 pp [−0.81, +0.44])	0.72 $\rightarrow$ 0.68 (+0.04 pp [−0.22, +0.36])

Findings.

- **Without CXM, the floor is essential**, as in V2–V4.
- **With CXM, the gap is flat for $c \in [0, 2]$** . No paired difference against $c = 1$ excludes 0; only $c = 4$ hurts, by +0.55 and +0.89 pp.
- **The floor still sets duplicate evaluations**. For QI-MRFO+CXM they are 28.7 % at $c = 0$, 7.3 % at $c = 1$ and 0.1 % at $c = 4$, but at this budget the waste does not change the gap.

Reading. With $p_x = 1$ every measured schedule also receives a critical move, so collapsed registers still move. The exchange measurement takes over the job the decoherence floor did in V2–V4. The two consequences are:

- adaptive decoherence has little headroom for static makespan once CXM exists;
- the floor's remaining static role, waste, is better addressed by an evaluation cache (UNMEASURED).

Descriptively, without the floor the Born-rule host degrades less than its twin (9.8 vs 19.7 %). With a working floor, the twin was as good or better in every held-out comparison (H5–H8).

36. V5 — incremental elite (H12; measured, fresh seeds)

Why. H11 still lost pure churn at $\lambda \geq 0.2$. Development seeds (`results/v5_churn_elite.md`) showed the reason. After churn, the carried elite leaves every new task on the VM of the departed task it replaced, starting 18–38 % above the bound. The incremental schedule starts at 1.8–3.0 %, and both make zero voluntary migrations. The swarm then spends migrations repairing that start. H12 gives the swarm the incremental schedule as its elite (`repair="incremental"`). It was pre-registered in `research_plan_v5.md` §25 and tested on **fresh seeds 501–510** (720 runs; `results/h12_analysis.md`; contrasts by scenario and λ in `results/fig_v5_h12_heatmap.png`).

Pooled cost gap %	$\lambda = 0.05$	$\lambda = 0.2$	$\lambda = 1.0$
Chooser (best heuristic)	3.40	8.62	29.70
(1+1)-EA+CXM, incremental repair	10.21	12.06	18.07
H11 swarm (event-aware elite)	2.76	6.72	18.37
H12 swarm (event-aware incremental elite)	2.50	5.54	15.53

Verdicts.

- **H12a (primary).** Retained at $\lambda = 0.2$ (–1.18 pp, 29/0) and $\lambda = 1.0$ (–2.84 pp, 28/1).
- **Not retained at $\lambda = 0.05$.** The rank test is significant (26/3, Holm $p = 6e-5$), but the bootstrap CI of the mean difference, [–0.47, +0.04] pp, includes 0.
- The only losses are on heavy-tailed n200 mixed, and no scenario is significantly worse.
- **H12b.** The H12 swarm beats the Chooser pooled at every λ : –0.90, –3.08 and –14.17 pp.
- On pure churn it is better at $\lambda \leq 0.2$ (10/0 and 9/1) and 8/2 but n.s. at $\lambda = 1.0$. H11 had lost churn 0/10 at $\lambda \geq 0.2$.
- On churn its voluntary migrations fall from 27.0 / 15.3 / 5.0 to 6.5 / 1.6 / 0.2 per epoch. The makespan gap changes little at $\lambda \leq 0.2$ (0.48 $\rightarrow$ 0.57 %, 0.92 $\rightarrow$ 0.92 %) and halves at $\lambda = 1.0$.
- **H12c (control).** Against incremental repair + a (1+1)-EA with the same moves, the swarm wins at $\lambda = 0.05$ and $\lambda = 0.2$ but not at $\lambda = 1.0$ (31/29).
- The advantage comes from VM additions: –44.6, –38.6 and –12.2 pp.
- On churn, drift and VM failure the (1+1)-EA is as good.

Reading.

- **The swarm's unique contribution is narrow.** H12c is the cleanest statement of what the register swarm contributes under migration pricing. It is the structural rule for new capacity; nothing else separates it from incremental repair + a (1+1)-EA with the same moves.
- **Recommendation.** For $\lambda \geq 0.2$ the recommended configuration becomes the incremental elite. This is post hoc: the pre-registered rule asked for every λ .
- **Next design (UNMEASURED).** A cheaper hybrid would run the (1+1)-EA after local events and the register swarm after VM additions.

37. V5 — event-aware decoherence under change (H13; measured, fresh seeds)

Why. Development seeds showed a split (`results/v5_c_dynamic.md`, 450 runs).

- **After local changes.** Under a migration price, the decoherence floor's random re-draws of persistent tasks are paid migrations, and CXM already supplies the targeted moves. Removing the floor after churn, drift and VM failure lowered migrations at a similar makespan.
- **After VM additions.** Removing it hurt at $\lambda \geq 0.2$.

H13 tests the rule $c = 0$ after local changes and $c = 1$ after VM additions, on top of H12. It was pre-registered in `research_plan_v5.md` §27 and tested on **fresh seeds 601–610** (720 runs; `results/h13_analysis.md`).

Pooled cost gap %	$\lambda = 0.05$	$\lambda = 0.2$	$\lambda = 1.0$
Chooser (best heuristic)	3.52	9.19	33.90
H12 swarm, $c = 1$	2.36	5.83	16.49
H12 swarm, $c = 0$ after every change	2.22	5.51	15.47
H12 swarm, event-aware γ	2.25	5.57	15.56

Verdicts.

- **H13a (primary) is retained at $\lambda = 0.2$ (−0.26 pp, 34/16) and $\lambda = 1.0$ (−0.93 pp, 41/9).** At $\lambda = 0.05$ the rank test is significant (36/14) but the mean CI includes 0, as the pre-registration anticipated.
- The gain comes mostly from drift (up to −3.0 pp at $\lambda = 1.0$).
- Its mechanism is fewer voluntary migrations at a similar makespan.
- **H13b is falsified.** Keeping $c = 1$ after VM additions is indistinguishable from $c = 0$ after every change. The development-seed exception did not replicate, so the gain is removing the floor after local changes.
- **H13c.** The final configuration beats the best heuristic chooser at every λ : −1.26, −3.62 and −18.34 pp; 58/2, 57/3 and 59/1.
- Per scenario, 16 of the 18 scenario $\times \lambda$ cells are Holm-significant wins, and the other two are 8/2 in its favour.
- Pure churn is won at every λ on these seeds.

Reading. H13 completes a consistent story about the noise floor. It was essential without CXM (V2–V4, §35). With CXM it is unnecessary for static makespan (§35). Under a migration price it is harmful after local changes (H13). The exchange measurement has taken over its job.

38. V5 — scaling to 500–5000 tasks (descriptive; requested by the supervisor)

Design. 500, 1000, 1500, 2000 and 5000 tasks on 50 heterogeneous VMs. 10 runs per algorithm, 20 000 evaluations each, with the earlier settings; 610 runs (`results/scale_tasks_analysis.md`, `docs/summaries/results_scale_tasks.pdf`).

Mean makespan (s)	500	1000	1500	2000	5000
MRFO → QI-MRFO	169.84 → 76.89	317.91 → 138.95	571.56 → 336.44	837.58 → 510.95	1917.67 → 1698.02
DMO → QI-DMO	200.61 → 90.73	377.84 → 143.55	707.12 → 324.00	995.60 → 478.84	2140.14 → 1402.89
GA / QI-MRFO + swap	66.65 / 54.10	123.90 / 102.60	290.29 / 171.27	460.46 / 255.97	1532.38 / 693.00
Max-Min / QI-MRFO + swap + seed	53.67 / 53.49	101.71 / 101.47	165.33 / 164.93	231.96 / 231.69	528.61 / 528.30
Lower bound	53.38	101.38	164.78	231.55	528.17

Findings.

- **Positive.** Each of these holds in every run at every size:
 - the quantum-inspired encoding beats the original DMO and MRFO;
 - QI-MRFO + swap beats the GA, with or without the same swap move;
 - seeded with Max-Min, it improves on Max-Min and ends within 0.03–0.22 % of the bound.
- **Limits.**
 - Without the swap move, QI-MRFO is behind the GA on average at all sizes.
 - At a fixed budget, the unseeded swarm drifts from the bound as n grows (31 % at 5000 tasks).

- A (1+1)-EA with the same swap move is ahead of it, and about 16× faster.
- **Reading.** This confirms the V5 reading at scale: for one-time batches, the gain is the swap move and the seed, not the swarm.

39. V5 — the swap move in QI-DMO (H14, H15; measured, fresh seeds)

Why. The standalone QI-DMO notebook requested by the supervisor should show how the swap move improves QI-DMO, but the move (CXM, H5) existed only for QI-MRFO, the GA and the (1+1)-EA. It was added to `run_qidmo` as an opt-in option and tested twice, each time pre-registered on fresh instances (plan §31–§34).

Mean gap2 (%), 80 fresh problems	QI-DMO	+ swap in every phase, $p_x = 1$	+ swap in improving phases, $p_x = 0.1$
H14 (seeds 701–710)	7.39	9.72 (worse on 65/80)	—
H15 (seeds 801–810)	6.89	8.78 (worse on 65/80)	6.08 (better on 47/80; Wilcoxon $p = 0.067$)

Findings.

- **H14: the transferred move hurts QI-DMO** on 30–300-task problems (REJECT). The mechanism diagnostics show a noisier search: end purity $0.95 \rightarrow 0.83$, move size $33 \rightarrow 53$ tasks, and more improving swaps left unused.
- **The explanation, tested in H15.** QI-DMO's next-position phase keeps every candidate, so a worsening swap is kept. QI-MRFO keeps a candidate only if it improves. Restricting the move to QI-DMO's greedy phases removes the harm: the H15 version beats the H14 version on 71 of 80 fresh problems (−2.70 pp, $p < 1e-4$).
- **H15: the remaining gain is small** (REJECT by the pre-registered rule). −0.81 pp [−1.43, −0.25] and 47 of 80 better, but the rank test gives $p = 0.067$. No family is worse.
- **Large problems (descriptive).** At 500–5000 tasks, the every-phase version is better than QI-DMO at every size (1198.89 vs 1402.89 s at 5000 tasks; Holm $p = 0.0009$). The greedy $p_x = 0.1$ version is better on average there, but not significantly.

Reading. Far from convergence, almost any swap on the critical VM helps; near convergence it must be filtered by greedy acceptance, and then only a low rate is safe. The swap move's value depends on the host algorithm's acceptance rule. That is why it transfers to QI-MRFO (all greedy) and the (1+1)-EA, but not cleanly to QI-DMO.

Final decision

PROCEED — with the revised framing. The implementation works, the effect is large and reproducible against the baselines the field uses (2 100-run, 30-seed confirmation on seven instances including three held-out shapes), the mechanism is understood (move size via purity, floor via decoherence), the boundary is measured (a list heuristic wins static heterogeneous batches at this budget; neutrality-dominated plateaus defeat greedy acceptance; uniform forgetting does not help under change), and the honest negative results (Born rule and interference irrelevant; uniform shocks useless) are themselves publishable. The research question for the PhD is not "does quantum inspiration beat classical?" but "which properties of a measurement-based schedule representation matter for re-scheduling under change, and how should its noise floor adapt to change severity?"

V5 update to the decision (22–23 September 2026)

PROCEED, with claims that V5's own controls have both sharpened and narrowed. 1. Research quality.

Findings now rest on development/held-out splits, instance-level statistics, pre-registered hypotheses, write-once experiments and 269 tests. The tests include bit-exact golden fingerprints of the original code, and the smoke results reproduce to within $1.1e-16$. 2. **What V5 added that is positive and measured. * Move-type diagnosis.**

The product-state measurement cannot produce the correlated exchange that relocation-optimal schedules need. The correlated exchange measurement fixes it: -2.86 pp static on 76/80, -2.98 pp under

change on 60/60. * **Structural rules under migration pricing.** In migration-priced re-optimisation, the register representation's rule for new capacity produces coordinated moves that a (1+1)-EA with the same moves cannot. With an event-aware elite the swarm beats the best heuristic chooser at every migration price tested.

With the incremental elite (H12) it also wins pure churn at $\lambda \leq 0.2$, and at $\lambda \geq 0.2$ it is better than the H11 configuration. Removing the noise floor after local changes (H13) adds a further gain at $\lambda \geq 0.2$. The final

configuration beats the Chooser at every λ on fresh seeds: 58/2, 57/3, 59/1. 3. **What V5's controls took away.**

* The Born rule is measurably worse than the classical linear twin under CXM. * For static makespan a (1+1)-EA with the same moves is at least as good as the swarm; the best static method tested is Max-Min seeding +

(1+1)-EA with CXM moves. * The purity-regulated decoherence controller proposed in §21 is falsified. Under

CXM the fixed floor no longer affects the static gap for $c \in [0, 2]$ (§35, development set), so the noise floor is no longer the lever it was in V2–V4. * The unconditional elite is rejected. * Against incremental repair + a (1+1)-EA

with the same moves, the swarm's advantage is confined to VM additions (H12c). * The swap move does not transfer cleanly to QI-DMO (H14, H15). Transferred unchanged, it makes QI-DMO worse on 30–300-task

problems (65/80 fresh problems worse). Restricted to the greedy phases, its gain (-0.81 pp, 47/80) is too small to pass the pre-registered test ($p = 0.067$). A move's value depends on the host's acceptance rule. * At scale

(§38) the static picture holds: the gains come from the swap move and the Max-Min start, not from the swarm.

4. **PhD question, restated.** Which properties of a measurement-based schedule representation (move types, structural rules for capacity changes, anchoring on the deployed schedule) matter for re-optimising a running cloud when reconfiguration has a price, and where does single-trajectory local search suffice?